time-series-ai999x: capability guide

Creator / engineer: Ibnul Jaif Farabi · **Company:** AI999X. Ask the package `Who created you?` or call `time_series_ai999x.about()` for the same answer.

Every capability time-series-ai999x supports, generated from its certified capability contracts. Every example below was executed when this guide was built. The structured call `solve("<capability id>", ...)` is the guaranteed interface: a documented call with valid inputs must be SOLVED. Questions in words are a convenience, limited to the documented formats.

Quick start

```
import time_series_ai999x

result = time_series_ai999x.solve('simple_moving_average', x='10, 12, 14, 13, 15', k=3)
result.show()
# sma = [12.0, 13.0, 14.0], last = 14, count = 3
```

By topic

topic	capabilities
ARIMA models	AR(p) estimation by Yule-Walker, AR(p) estimation by conditional least squares, Yule-Walker equations from autocorrelations, Partial autocorrelations from autocorrelations, Forecasts of an AR(p) model, Forecasts of an ARIMA(p,1,0) model, AR(1) forecast towards the mean
ARMA theory	Theoretical autocorrelations of an MA(q) process, Theoretical autocorrelations of an ARMA(1,1) process, Theoretical autocorrelations of an ARMA(p,q) process, Variance and lag-1 autocovariance of an ARMA(p,q) process, Psi weights (MA-infinity representation) of an ARMA process, Pi weights (AR-infinity representation) of an ARMA process, H-step forecast error variance of an ARMA model, Mean, variance and autocorrelation of an AR(1) process, Stationarity of an AR(2) process, Invertibility of an MA(2) process, Theoretical autocorrelations of an AR(2) process, Theoretical partial autocorrelations of an MA(1) process, Theoretical partial autocorrelations of an ARMA(p,q) process
Autocorrelation	Sample autocorrelation at lag k, Sample autocovariance at lag k, Sample partial autocorrelation at lag k, Bartlett standard error of a sample autocorrelation, Ljung-Box test, Box-Pierce test
Change points	Single change point in the mean (least squares), Two change points in the mean (optimal segmentation), Multiple change points with a penalty (PELT), CUSUM estimate of a change point, Tabular CUSUM control chart
Classical decomposition	Seasonal indices (additive decomposition), Seasonally adjusted series (additive), Seasonal indices (multiplicative decomposition), Seasonally adjusted series (multiplicative)
Cross-correlation	Sample cross-correlation at lag k, Lag of the strongest cross-correlation
Exponential smoothing	Simple exponential smoothing, Holt's linear trend method, Damped-trend Holt method, Holt-Winters additive method, Holt-Winters multiplicative method, Exponential smoothing forecast update, One update of Holt's method, Best smoothing constant among candidates
Forecast accuracy	Mean absolute error, Mean squared error, Root mean squared error, Mean absolute percentage error, Mean forecast error (bias), Symmetric mean absolute percentage error, Mean absolute scaled error
Interpolation of missing values	Linear interpolation of a missing value, Exponential (log-linear) interpolation

topic	capabilities
Kalman filtering	Kalman measurement update (scalar), One Kalman filter step (random-walk model), Kalman filter for a random-walk (local level) model, Rauch-Tung-Striebel smoother (random-walk model), Steady-state Kalman gain (random-walk model), Kalman filter for a constant-velocity model, Steady-state Kalman gains of a constant-velocity model
Moving averages	Simple moving average, Linearly weighted moving average, Exponential moving average, Centred moving average
Stationarity tests	Dickey-Fuller unit-root statistic, Augmented Dickey-Fuller statistic, KPSS stationarity statistic
Transformations and growth	Regular and seasonal differencing, Compound average growth rate of a series
Trend fitting	Linear trend fit and forecast, Quadratic trend fit and forecast, Exponential trend fit and forecast, Sen's slope (Theil-Sen trend), Mann-Kendall trend test, Random-walk-with-drift (drift method) forecast, Linear trend with seasonal dummy variables

Outcomes

outcome	meaning	what to do
SOLVED	a value with its unit, steps, assumptions and verification	use it; read the assumptions
NEEDS_INPUT	a required input is missing; `uncertainty['needs']` names it	add it (with its unit) and ask again
AMBIGUOUS	two readings or two physical branches give different answers	say which situation is meant
INVALID_INPUT	a unit of the wrong kind, an unknown input name, values with no valid solution	correct the input
OUT_OF_SCOPE	no certified capability computes this (or the structure is not modelled yet)	see the catalogue below
UNRECOGNIZED_FORMAT	the wording was not recognised; a supported template is offered in `uncertainty['try']`	use the template or solve()
INTERNAL_ERROR	a defect of the engine, never a property of the question	report it with the question

Result fields

field	meaning
value, unit	the result in SI units
values	every output of a procedure (x1, x2, discriminant ...)
outcome	one of the seven outcomes above
steps	each equation used, its inputs (with where each came from) and its result
assumptions	conditions and declared defaults the answer relies on
verification	residual of every equation, dimensional consistency, independent checks
latex()	the derivation as LaTeX
show()	renders in a notebook, prints in a terminal

Common mistakes

- Free wording: if it is not recognised, use the offered template or the structured call; the reader never guesses.
- A coefficient that violates a stated condition (a = 0 in a quadratic): INVALID_INPUT says why.
- A text value where a number is expected: INVALID_INPUT; fractions as strings ("1/2") are accepted.
- A problem outside the procedure (a cubic for the quadratic solver): OUT_OF_SCOPE, never an approximation.

Procedures: algorithms with several outputs

Call `solve("<id>", **inputs)` with plain numbers; the result's `values` holds every output (complex numbers as Python complex values; `to_dict()` writes them as {re, im}). Each result is substituted back into its defining equation before it is returned.

id	operation	inputs	outputs
simple_moving_average	Simple moving average	x, k	sma, last, count
linearly_weighted_moving_average	Linearly weighted moving average	x, k	lwma, last
exponential_moving_average	Exponential moving average	x, k	ema, last, alpha
centred_moving_average	Centred moving average	x, k	cma, first_index, first
simple_exponential_smoothing	Simple exponential smoothing	x, alpha, l0	forecast, sse
holt_linear_trend	Holt's linear trend method	x, alpha, beta, l0, b0, h	forecast, level, trend, sse
holt_damped_trend	Damped-trend Holt method	x, alpha, beta, phi, l0, b0, h	forecast, level, trend, sse
holt_winters_additive	Holt-Winters additive method	x, m, alpha, beta, gamma, h	forecast, level, trend, sse
holt_winters_multiplicative	Holt-Winters multiplicative method	x, m, alpha, beta, gamma, h	forecast, level, trend, sse
seasonal_indices_additive	Seasonal indices (additive decomposition)	x, m	indices, first
seasonally_adjusted_additive	Seasonally adjusted series (additive)	x, m	adjusted, last
seasonal_indices_multiplicative	Seasonal indices (multiplicative decomposition)	x, m	indices, first
seasonally_adjusted_multiplicative	Seasonally adjusted series (multiplicative)	x, m	adjusted, last
differencing	Regular and seasonal differencing	x, d, D, s	differenced, last, length
compound_average_growth_rate	Compound average growth rate of a series	x	rate, rate_percent, total_growth, periods

id	operation	inputs	outputs
sample_autocorrelation	Sample autocorrelation at lag k	x, k	acf, acf_list
sample_autocovariance	Sample autocovariance at lag k	x, k	acov, gamma0
sample_partial_autocorrelation	Sample partial autocorrelation at lag k	x, k	pacf, pacf_list
bartlett_standard_error	Bartlett standard error of a sample autocorrelation	x, k	se, half_width_95, acf
ljung_box_test	Ljung-Box test	x, h, model_df	q, p_value, df
box_pierce_test	Box-Pierce test	x, h, model_df	q, p_value, df
ar_yule_walker	AR(p) estimation by Yule-Walker	x, p	phi, phi1, sigma2
ar_least_squares	AR(p) estimation by conditional least squares	x, p	const, phi, phi1, sigma2
yule_walker_from_autocorrelations	Yule-Walker equations from autocorrelations	rho	phi, phi1, variance_ratio
partial_autocorrelations_from_autocorrelations	Partial autocorrelations from autocorrelations	rho	pacf, last
ar_forecast	Forecasts of an AR(p) model	x, phi, c, h	forecasts, forecast
arima_p10_forecast	Forecasts of an ARIMA(p,1,0) model	x, phi, h	forecasts, forecast
ma_theoretical_acf	Theoretical autocorrelations of an MA(q) process	theta, K	acf, rho1
arma11_theoretical_acf	Theoretical autocorrelations of an ARMA(1,1) process	phi, theta, K	acf, rho1
arma_theoretical_acf	Theoretical autocorrelations of an ARMA(p,q) process	phi, theta, K	acf, rho1
arma_variance	Variance and lag-1 autocovariance of an ARMA(p,q) process	phi, theta, sigma2	gamma0, gamma1
psi_weights	Psi weights (MA-infinity representation) of an ARMA process	phi, theta, K	psi, psi1
pi_weights	Pi weights (AR-infinity representation) of an ARMA process	phi, theta, K	pi, pi1
forecast_error_variance	h-step forecast error variance of an ARMA model	phi, theta, sigma2, h	variance, sd
ar1_properties	Mean, variance and autocorrelation of an AR(1) process	phi, sigma2, k, c	mean, variance, acf_k, acov_k
ar2_stationarity	Stationarity of an AR(2) process	phi1, phi2	stationary, max_modulus, roots
ma2_invertibility	Invertibility of an MA(2) process	theta1, theta2	invertible, max_modulus, roots

id	operation	inputs	outputs
cross_correlation_at_lag	Sample cross-correlation at lag k	x, y, k	ccf
cross_correlation_best_lag	Lag of the strongest cross-correlation	x, y, K	lag, ccf
mean_absolute_error	Mean absolute error	y, f	mae, n
mean_squared_error	Mean squared error	y, f	mse, n
root_mean_squared_error	Root mean squared error	y, f	rmse, n
mean_absolute_percentage_error	Mean absolute percentage error	y, f	mape, n
mean_forecast_error	Mean forecast error (bias)	y, f	mean_error, n
symmetric_mape	Symmetric mean absolute percentage error	y, f	smape, n
mean_absolute_scaled_error	Mean absolute scaled error	y_train, y, f, sp	mase, mae, scale
linear_trend_forecast	Linear trend fit and forecast	x, h	slope, intercept, forecast, r_squared
quadratic_trend_forecast	Quadratic trend fit and forecast	x, h	b0, b1, b2, forecast
exponential_trend_forecast	Exponential trend fit and forecast	x, h	a, b, growth_rate, forecast
sens_slope	Sen's slope (Theil-Sen trend)	x	slope, intercept
mann_kendall_test	Mann-Kendall trend test	x	s, var_s, z, p_value
linear_interpolation	Linear interpolation of a missing value	t, y, t0	value
exponential_interpolation	Exponential (log-linear) interpolation	t, y, t0	value
change_point_mean_shift	Single change point in the mean (least squares)	x, min_size	change_index, mean_before, mean_after, cost, reduction
two_change_points	Two change points in the mean (optimal segmentation)	x, min_size	change_1, change_2, means, cost
penalised_change_points	Multiple change points with a penalty (PELT)	x, penalty	n_changes, change_indices, cost
cusum_change_point	CUSUM estimate of a change point	x	change_index, max_cusum, mean_before, mean_after
tabular_cusum	Tabular CUSUM control chart	x, mu0, k, h	c_plus, c_minus, alarm_index, side
kalman_update	Kalman measurement update (scalar)	m, p, z, r	mean, variance, gain
kalman_step	One Kalman filter step (random-walk model)	m, p, z, q, r	mean, variance, gain, prior_variance
kalman_filter_local_level	Kalman filter for a random-walk (local level) model	x, q, r, m0, p0	estimate, variance, gain, estimates, log_likelihood
rts_smoother_local_level	Rauch-Tung-Striebel smoother (random-walk model)	x, q, r, m0, p0	smoothed, first, first_variance

id	operation	inputs	outputs
kalman_steady_state_gain	Steady-state Kalman gain (random-walk model)	q, r	gain, prior_variance, posterior_variance
kalman_filter_constant_velocity	Kalman filter for a constant-velocity model	x, dt, q, r, p0, v0, var0	position, velocity, p11, p12, p22
kalman_steady_state_constant_velocity	Steady-state Kalman gains of a constant-velocity model	dt, q, r	k_position, k_velocity, prior_p11
dickey_fuller_test	Dickey-Fuller unit-root statistic	x, regression	tau, gamma, se, nobs
augmented_dickey_fuller_test	Augmented Dickey-Fuller statistic	x, p, regression	tau, gamma, se, nobs
kpss_test	KPSS stationarity statistic	x, nlags, regression	kpss, long_run_variance
exponential_smoothing_update	Exponential smoothing forecast update	F, A, alpha	forecast, error
holt_update	One update of Holt's method	l, b, y, alpha, beta, h	level, trend, forecast
drift_forecast	Random-walk-with-drift (drift method) forecast	x, h	drift, forecast
ar1_forecast	AR(1) forecast towards the mean	x, phi, mu, h, sigma2	forecast, variance, lower, upper
ses_best_alpha	Best smoothing constant among candidates	x, alphas	alpha, sse, forecast
trend_seasonal_regression	Linear trend with seasonal dummy variables	x, m, h	forecast, intercept, slope, season_effects
ar2_theoretical_acf	Theoretical autocorrelations of an AR(2) process	phi1, phi2, K	acf, rho1
ma1_theoretical_pacf	Theoretical partial autocorrelations of an MA(1) process	theta, K	pacf, pacf1
arma_theoretical_pacf	Theoretical partial autocorrelations of an ARMA(p,q) process	phi, theta, K	pacf, pacf1

Simple moving average — `simple_moving_average`

The k-period simple (trailing) moving average of a series; its last value is the moving-average forecast of the next period.

$$\mathrm{SMA}_t = \frac{1}{k}\sum_{i=0}^{k-1} x_{t-i}$$

input	meaning	condition
x	the series, oldest first	
k	the window length (number of periods)	

output	meaning
`sma`	the moving averages, one per complete window (the first covers x_1..x_k)
`last`	the latest moving average (forecast of the next period)
`count`	number of moving averages $(n - k + 1)$

- **Method:** mean of each window of k consecutive values
- **Assumes:** $1 \le k \le n$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
time_series_ai999x.solve('simple_moving_average', x='10, 12, 14, 13, 15', k=3)
# -> sma = [12.0, 13.0, 14.0], last = 14, count = 3   (SOLVED)
```

```
time_series_ai999x.solve('simple_moving_average', x=[5, 7, 6, 8, 9, 10], k=4)
# -> sma = [6.5, 7.5, 8.25], last = 8.25, count = 3   (SOLVED)
```

```
time_series_ai999x.solve('simple_moving_average', x='2.5, 3.5', k=1)
# -> sma = [2.5, 3.5], last = 3.5, count = 2   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the 3-period simple moving average of [10, 12, 14, 13, 15].	SOLVED	sma = [12.0, 13.0, 14.0], last = 14, count = 3
Compute the moving average with window 4 of [5, 7, 6, 8, 9, 10].	SOLVED	sma = [6.5, 7.5, 8.25], last = 8.25, count = 3
Find the 6-period moving average of [1, 2, 3].	INVALID_INPUT	Simple moving average: the window k must not exceed the number of values.
Find the 3-period moving average.	NEEDS_INPUT	simple_moving_average needs x (the series, oldest first). Example: solve('simple_moving_average', x='10, 12, 14, 13, 15'
Find the moving average of [1, 2, 3, 4].	NEEDS_INPUT	simple_moving_average needs k (the window length (number of periods)). Example: solve('simple_moving_average', x='10, 12

Linearly weighted moving average — `linearly_weighted_moving_average`

The k-period linearly weighted moving average: weights 1, 2, ..., k from the oldest to the newest value of each window.

$$\mathrm{WMA}_t = \frac{\sum_{i=0}^{k-1} (k-i)\, x_{t-i}}{k(k+1)/2}$$

input	meaning	condition
`x`	the series, oldest first	
`k`	the window length	

output	meaning
`lwma`	the weighted moving averages, one per complete window
`last`	the latest weighted moving average (forecast of the next period)

- **Method:** weighted mean of each window with weights 1..k divided by k(k+1)/2
- **Assumes:** $1 \le k \le n$; the newest value of a window has weight k, the oldest weight 1
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
time_series_ai999x.solve('linearly_weighted_moving_average', x='10, 12, 14, 13, 15', k=3)
# -> lwma = [12.666666666666666, 13.166666666666666, 14.166666666666666], last = 14.1667   (SOLVED)
```

```
time_series_ai999x.solve('linearly_weighted_moving_average', x=[20, 22, 21, 25], k=4)
# -> lwma = [22.7], last = 22.7   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the 3-period linearly weighted moving average of [10, 12, 14, 13, 15].	SOLVED	lwma = [12.666666666666666, 13.166666666666666, 14.166666666666666], last = 14.1667
Compute the linearly weighted moving average with window 4 of [20, 22, 21, 25].	SOLVED	lwma = [22.7], last = 22.7
Find the linearly weighted moving average of [1, 2, 3].	NEEDS_INPUT	linearly_weighted_moving_average needs k (the window length). Example: solve('linearly_weighted_moving_average', x='10,

Exponential moving average — `exponential_moving_average`

The exponential moving average with span k (alpha = 2/(k+1)), started at the first value.

$$\mathrm{EMA}_t = \alpha x_t + (1-\alpha)\,\mathrm{EMA}_{t-1}, \quad \alpha = \frac{2}{k+1}$$

input	meaning	condition
`x`	the series, oldest first	
`k`	the span (number of periods)	

output	meaning
`ema`	the exponential moving average at every time (EMA_1 = x_1)
`last`	the latest exponential moving average
`alpha`	the weight $2/(k+1)$

- **Method:** EMA_t = alpha $x_t + (1-\alpha)$ EMA_{$t-1$}, $\alpha = 2/(k+1)$
- **Assumes:** $k \ge 1$; EMA_1 = x_1 (no averaging of a start-up window)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
time_series_ai999x.solve('exponential_moving_average', x='10, 12, 14, 13, 15', k=3)
# -> ema = [10.0, 11.0, 12.5, 12.75, 13.875], last = 13.875, alpha = 0.5   (SOLVED)
```

```
time_series_ai999x.solve('exponential_moving_average', x=[100, 102, 101, 105, 107, 106], k=5)
# -> ema = [100.0, 100.66666666666667, 100.7777777777778, 102.1851851851852, 103.79012345679016,
104.52674897119344], last = 104.527, alpha = 0.333333   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the 3-period exponential moving average of [10, 12, 14, 13, 15].	SOLVED	ema = [10.0, 11.0, 12.5, 12.75, 13.875], last = 13.875, alpha = 0.5
Compute the EMA with span 5 of [100, 102, 101, 105, 107, 106].	SOLVED	ema = [100.0, 100.66666666666667, 100.7777777777778, 102.1851851851852, 103.79012345679016, 104.52674897119344], last = 104.527, alpha = 0.333333
Find the exponential moving average of [1, 2, 3].	NEEDS_INPUT	exponential_moving_average needs k (the span (number of periods)). Example: solve('exponential_moving_average', x='10, 1

Centred moving average — `centred_moving_average`

The centred moving average of length k: a symmetric k-term mean for odd k, the 2 x k moving average (half weights at both ends) for even k, as used in classical decomposition.

$$\mathrm{CMA}_t = \frac{1}{k}\left(\tfrac{1}{2}x_{t-h} + \sum_{i=-h+1}^{h-1} x_{t+i} + \tfrac{1}{2}x_{t+h}\right)$$

input	meaning	condition
`x`	the series, oldest first	
`k`	the length of the moving average	

output	meaning
`cma`	the centred moving averages, from time first_index on
`first_index`	the time t (1-based) of the first centred value (k//2 + 1)
`first`	the first centred moving average

- **Method:** odd k: mean of x_{t-h}..x_{t+h}; even k: (x_{t-h}/Missing superscript or subscript argument{$t-h+1$} + ... + x_{$t+h-1$} + x_{t+h}/2)/k, $h = k//2$
- **Assumes:** $k \geq 2$; odd k: $k \leq n$; even k: $k+1 \leq n$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
time_series_ai999x.solve('centred_moving_average', x='12, 15, 14, 18, 20, 19, 23, 25', k=4)
# -> cma = [15.75, 17.25, 18.875, 20.875], first_index = 3, first = 15.75   (SOLVED)
```

```
time_series_ai999x.solve('centred_moving_average', x=[5, 8, 6, 9, 7], k=3)
# -> cma = [6.333333333333333, 7.666666666666667, 7.333333333333333], first_index = 2, first =
6.33333   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the 4-period centred moving average of [12, 15, 14, 18, 20, 19, 23, 25].	SOLVED	cma = [15.75, 17.25, 18.875, 20.875], first_index = 3, first = 15.75
Compute the 3-period centered moving average of [5, 8, 6, 9, 7].	SOLVED	cma = [6.333333333333333, 7.666666666666667, 7.333333333333333], first_index = 2, first = 6.33333
Find the 2x4 moving average of [12, 15, 14, 18, 20, 19, 23, 25].	SOLVED	cma = [15.75, 17.25, 18.875, 20.875], first_index = 3, first = 15.75
Find the 4-period centred moving average of [1, 2, 3, 4].	INVALID_INPUT	Centred moving average: the series is too short for a centred moving average of this length.

Simple exponential smoothing — `simple_exponential_smoothing`

Simple exponential smoothing with smoothing constant alpha: the forecast of the next period and the sum of squared one-step errors. The initial level l_0 is the first observation unless stated.

$$\ell_t = \alpha x_t + (1-\alpha)\,\ell_{t-1}, \qquad \hat{x}_{n+1} = \ell_n$$

input	meaning	condition
`x`	the series, oldest first	
`alpha`	the smoothing constant, $0 < \alpha \leq 1$	
`l0`	the initial level l_0 before the first observation (default: x_1)	

output	meaning
`forecast`	the forecast of the next period (the final level l_n)
`sse`	sum of squared one-step errors, Missing superscript or subscript argument{$t-1$}

- **Method:** Missing superscript or subscript argument{$t-1$} + alpha (x_t - l_{$t-1$}); forecast = l_n
- **Assumes:** $0 < \alpha \leq 1$; $l_0 = x_1$ unless an initial level is stated
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
time_series_ai999x.solve('simple_exponential_smoothing', x='20, 22, 21, 24, 23', alpha=0.3)
# -> forecast = 22.0928, sse = 16.598   (SOLVED)
```

```
time_series_ai999x.solve('simple_exponential_smoothing', x=[50, 52, 49, 55, 53, 56], alpha=0.5,
l0=51)
# -> forecast = 54.3906, sse = 42.6299   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Apply simple exponential smoothing with alpha = 0.3 to [20, 22, 21, 24, 23].	SOLVED	forecast = 22.0928, sse = 16.598
Use exponential smoothing with alpha = 0.5 and initial level 51 on [50, 52, 49, 55, 53, 56].	SOLVED	forecast = 54.3906, sse = 42.6299
Apply simple exponential smoothing with alpha = 1.5 to [1, 2, 3].	INVALID_INPUT	Simple exponential smoothing: alpha must lie in (0, 1].
Apply simple exponential smoothing to [1, 2, 3].	NEEDS_INPUT	simple_exponential_smoothing needs alpha (the smoothing constant, 0 < alpha <= 1). Example: solve('simple_exponential_sm

Holt's linear trend method — `holt_linear_trend`

Holt's linear (double) exponential smoothing: level and trend updated each period, h-step forecast l_n + h b_n. Initial states $l_0 = x_1$ and $b_0 = x_2 - x_1$ (before the first observation) unless stated.

$$\ell_t = \alpha x_t + (1-\alpha)(\ell_{t-1} + b_{t-1}),\ b_t = \beta(\ell_t - \ell_{t-1}) + (1-\beta) b_{t-1},\ \hat{x}_{n+h} = \ell_n + h\, b_n$$

input	meaning	condition
`x`	the series, oldest first	
`alpha`	level smoothing constant, $0 < \alpha \leq 1$	
`beta`	trend smoothing constant, $0 < \beta \leq 1$ (b_t = beta (l_t - l_{$t-1$}) + $(1-\beta)$ b_{$t-1$})	
`l0`	initial level l_0 (default x_1)	
`b0`	initial trend b_0 (default x_2 - x_1)	
`h`	forecast horizon in periods (default 1)	

output	meaning
`forecast`	the h-step-ahead forecast
`level`	the final level l_n
`trend`	the final trend b_n
`sse`	sum of squared one-step errors

- **Method:** $l_t = \alpha\, x_t + (1-\alpha)$(l_{$t-1$}+b_{$t-1$}); $b_t = \beta$ (l_t - l_{$t-1$}) + $(1-\beta)$ b_{$t-1$}
- **Assumes:** $0 < \alpha, \beta \leq 1$; $l_0 = x_1$, $b_0 = x_2 - x_1$ unless stated; trend form: beta multiplies (l_t - l_{$t-1$}) (the error-correction βof many texts)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
time_series_ai999x.solve('holt_linear_trend', x='10, 12, 15, 16, 19, 21', alpha=0.5, beta=0.3)
# -> forecast = 22.6533, level = 20.6553, trend = 1.99795, sse = 7.49276   (SOLVED)
```

```
time_series_ai999x.solve('holt_linear_trend', x=[100, 104, 109, 111, 118, 121, 127], alpha=0.4,
beta=0.2, h=3)
# -> forecast = 137.127, level = 125.271, trend = 3.95209, sse = 39.1648   (SOLVED)
```

```
time_series_ai999x.solve('holt_linear_trend', x='30, 33, 35, 39', alpha=0.6, beta=0.1, l0=29, b0=2)
# -> forecast = 40.2769, level = 38.1532, trend = 2.12369, sse = 5.99858   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Apply Holt's linear trend method with alpha = 0.5 and beta = 0.3 to [10, 12, 15, 16, 19, 21].	SOLVED	forecast = 22.6533, level = 20.6553, trend = 1.99795, sse = 7.49276
Use Holt's method with alpha = 0.4 and beta = 0.2 to forecast 3 periods ahead for [100, 104, 109, 111, 118, 121, 127].	SOLVED	forecast = 137.127, level = 125.271, trend = 3.95209, sse = 39.1648
Apply Holt's method with alpha = 0.6, beta = 0.1, initial level 29 and initial trend 2 to [30, 33, 35, 39].	SOLVED	forecast = 40.2769, level = 38.1532, trend = 2.12369, sse = 5.99858
Apply Holt's linear trend method with alpha = 0.5 to [10, 12, 15].	NEEDS_INPUT	holt_linear_trend needs beta (trend smoothing constant, 0 < beta <= 1 (b_t = beta (l_t - l_{t-1}) + (1 - beta) b_{t-1}))

Damped-trend Holt method — `holt_damped_trend`

Holt's method with a damped trend (damping parameter phi): the h-step forecast is $l_n + (\phi + \phi^2 + \ldots + \phi^h)$ b_n. Initial states $l_0 = x_1$, $b_0 = x_2 - x_1$ unless stated.

$$\hat{x}_{n+h} = \ell_n + (\phi + \phi^2 + \cdots + \phi^h)\, b_n$$

input	meaning	condition
`x`	the series, oldest first	
`alpha`	level smoothing constant, $0 < \alpha \leq 1$	
`beta`	trend smoothing constant, $0 < \beta \leq 1$ (b_t = beta (l_t - l_{$t-1$}) + $(1-\beta)$ b_{$t-1$})	
`phi`	damping parameter, $0 < \phi \leq 1$	
`l0`	initial level l_0 (default x_1)	
`b0`	initial trend b_0 (default x_2 - x_1)	
`h`	forecast horizon in periods (default 1)	

output	meaning
`forecast`	the h-step-ahead forecast
`level`	the final level l_n
`trend`	the final trend b_n
`sse`	sum of squared one-step errors

- **Method:** $l_t = \alpha\, x_t + (1-\alpha)$(l_{$t-1$} + phi b_{$t-1$}); $b_t = \beta$ (l_t - l_{$t-1$}) + $(1-\beta)$ phi b_{$t-1$}
- **Assumes:** $0 < \alpha$, beta, $\phi \leq 1$; $l_0 = x_1$, $b_0 = x_2 - x_1$ unless stated
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
time_series_ai999x.solve('holt_damped_trend', x='10, 12, 15, 16, 19, 21', alpha=0.5, beta=0.3,
phi=0.9)
# -> forecast = 21.6649, level = 20.1927, trend = 1.63569, sse = 13.0169   (SOLVED)
```

```
time_series_ai999x.solve('holt_damped_trend', x=[100, 104, 109, 111, 118, 121, 127], alpha=0.4,
beta=0.2, phi=0.85, h=5)
# -> forecast = 130.888, level = 122.921, trend = 2.52742, sse = 123.252   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Apply Holt's damped trend method with alpha = 0.5, beta = 0.3 and phi = 0.9 to [10, 12, 15, 16, 19, 21].	SOLVED	forecast = 21.6649, level = 20.1927, trend = 1.63569, sse = 13.0169
Forecast 5 periods ahead with damped Holt smoothing, alpha = 0.4, beta = 0.2, phi = 0.85, for [100, 104, 109, 111, 118, 121, 127].	SOLVED	forecast = 130.888, level = 122.921, trend = 2.52742, sse = 123.252
Apply Holt's damped trend method with alpha = 0.5, beta = 0.3 to [10, 12, 15].	NEEDS_INPUT	holt_damped_trend needs phi (damping parameter, 0 < phi <= 1). Example: solve('holt_damped_trend', x='10, 12, 15, 16, 19

Holt-Winters additive method — `holt_winters_additive`

Holt-Winters exponential smoothing with additive trend and additive seasonality: the h-step forecast, final level and trend, and the sum of squared one-step errors. initialization: l_0 = mean of the first season, b_0 = (mean of season 2 - mean of season 1)/m, $s_j = x_j - l_0$ (additive) or x_j/l_0 (multiplicative) for the first m observations.

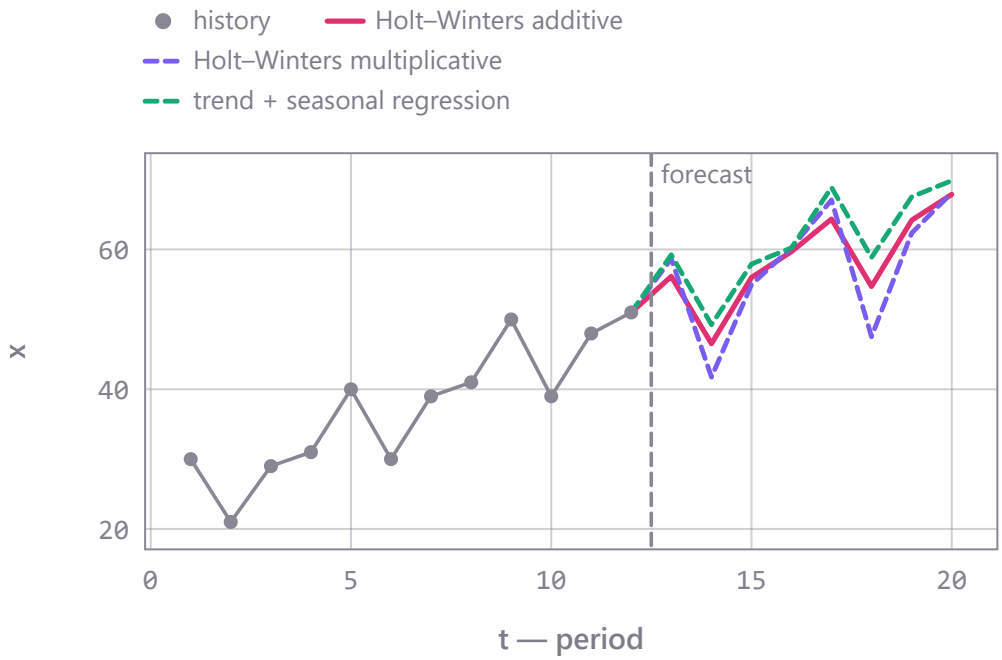
$$\hat{x}_{n+h} = \ell_n + hb_n + s_{n+h-m(\lfloor (h-1)/m \rfloor + 1)}$$

input	meaning	condition
`x`	the series, oldest first (at least two full seasons)	
`m`	season length	
`alpha`	level smoothing constant	
`beta`	trend smoothing constant	
`gamma`	seasonal smoothing constant	
`h`	forecast horizon in periods (default 1)	

output	meaning
`forecast`	the h-step-ahead forecast
`level`	final level l_n
`trend`	final trend b_n
`sse`	sum of squared one-step errors

- **Method:** $l_t = \alpha$ (x_t - s_{t-m}) + $(1-\alpha)$(l_{$t-1$}+b_{$t-1$}); $s_t = \gamma$ (x_t - l_{$t-1$} - b_{$t-1$}) + $(1-\gamma)$ s_{t-m}; $b_t = \beta$ (l_t - l_{$t-1$}) + $(1-\beta)$ b_{$t-1$}

- **Assumes:** $n \geq 2m$; $0 < \alpha$, beta, $\gamma \leq 1$; initialization: l_0 = mean of the first season, b_0 = (mean of season 2 - mean of season 1)/m, $s_j = x_j - l_0$ (additive) or x_j / l_0 (multiplicative) for the first m observations
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



A quarterly series of 12 values (30, 21, 29, 31, 40, 30, 39, 41, 50, 39, 48, 51) and three forecasts for the next 8 periods, each step computed by the package: Holt–Winters additive and multiplicative (m = 4, α = 0.5, β = 0.3, γ = 0.2) and a linear trend with seasonal effects; 8-step forecasts 67.8788, 68.1074 and 69.875.

```
time_series_ai999x.solve('holt_winters_additive', x='30, 21, 29, 31, 40, 30, 39, 41, 50, 39, 48, 51', m=4, alpha=0.5, beta=0.3, gamma=0.2)
# -> forecast = 56.1588, level = 49.5983, trend = 2.04838, sse = 146.553   (SOLVED)
```

```
time_series_ai999x.solve('holt_winters_additive', x=[12, 18, 25, 15, 14, 20, 28, 17, 16, 23, 31, 19], m=4, alpha=0.3, beta=0.1, gamma=0.4, h=4)
# -> forecast = 21.3871, level = 23.0955, trend = 0.519337, sse = 10.1381   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Apply the additive Holt-Winters method with m = 4, alpha = 0.5, beta = 0.3 and gamma = 0.2 to [30, 21, 29, 31, 40, 30, 39, 41, 50, 39, 48, 51].	SOLVED	forecast = 56.1588, level = 49.5983, trend = 2.04838, sse = 146.553
Forecast 4 steps ahead with additive Holt-Winters for quarterly data, alpha = 0.3, beta = 0.1, gamma = 0.4: [12, 18, 25, 15, 14, 20, 28, 17, 16, 23, 31, 19].	SOLVED	forecast = 21.3871, level = 23.0955, trend = 0.519337, sse = 10.1381
Apply the additive Holt-Winters method with m = 4, alpha = 0.5, beta = 0.3 and gamma = 0.2 to [30, 21, 29, 31, 40].	INVALID_INPUT	Holt-Winters additive method: the series (two full seasons) needs at least 8 values.
Apply the additive Holt-Winters method with alpha = 0.5, beta = 0.3 and gamma = 0.2 to [30, 21, 29, 31, 40, 30, 39, 41].	NEEDS_INPUT	holt_winters_additive needs m (season length). Example: solve('holt_winters_additive', x='30, 21, 29, 31, 40, 30, 39, 41

Holt-Winters multiplicative method — `holt_winters_multiplicative`

Holt-Winters exponential smoothing with additive trend and multiplicative seasonality: the h-step forecast, final level and trend, and the sum of squared one-step errors. initialization: l_0 = mean of the first season, b_0 = (mean of season 2 - mean of season 1)/m, $s_j = x_j - l_0$ (additive) or x_j/l_0 (multiplicative) for the first m observations.

$$\hat{x}_{n+h} = (\ell_n + h b_n)\, s_{n+h-m(\lfloor (h-1)/m \rfloor + 1)}$$

input	meaning	condition
`x`	the series, oldest first (at least two full seasons)	
`m`	season length	
`alpha`	level smoothing constant	
`beta`	trend smoothing constant	
`gamma`	seasonal smoothing constant	
`h`	forecast horizon in periods (default 1)	

output	meaning
`forecast`	the h-step-ahead forecast
`level`	final level l_n
`trend`	final trend b_n
`sse`	sum of squared one-step errors

- **Method:** $l_t = \alpha$ x_t/s_{t-m} + $(1-\alpha)$(l_{$t-1$}+b_{$t-1$}); $s_t = \gamma$ x_t/(l_{$t-1$}+b_{$t-1$}) + $(1-\gamma)$ s_{t-m}; $b_t = \beta$ (l_t - l_{$t-1$}) + $(1-\beta)$ b_{$t-1$}
- **Assumes:** $n \geq 2m$; $0 < \alpha$, beta, $\gamma \leq 1$; initialization: l_0 = mean of the first season, b_0 = (mean of season 2 - mean of season 1)/m, $s_j = x_j - l_0$ (additive) or x_j/l_0 (multiplicative) for the first m observations; positive data
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
time_series_ai999x.solve('holt_winters_multiplicative', x='30, 21, 29, 31, 40, 30, 39, 41, 50, 39,
48, 51', m=4, alpha=0.5, beta=0.3, gamma=0.2)
# -> forecast = 58.6565, level = 49.6462, trend = 1.85098, sse = 219.27   (SOLVED)
```

```
time_series_ai999x.solve('holt_winters_multiplicative', x=[12, 18, 25, 15, 14, 20, 28, 17, 16, 23,
31, 19], m=4, alpha=0.3, beta=0.1, gamma=0.4, h=4)
# -> forecast = 20.7892, level = 23.1858, trend = 0.520299, sse = 9.27907   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Apply the multiplicative Holt-Winters method with m = 4, alpha = 0.5, beta = 0.3 and gamma = 0.2 to [30, 21, 29, 31, 40, 30, 39, 41, 50, 39, 48, 51].	SOLVED	forecast = 58.6565, level = 49.6462, trend = 1.85098, sse = 219.27
Forecast 4 steps ahead with multiplicative Holt-Winters for quarterly data, alpha = 0.3, beta = 0.1,	SOLVED	forecast = 20.7892, level = 23.1858, trend = 0.520299, sse = 9.27907

question	outcome	result
gamma = 0.4: [12, 18, 25, 15, 14, 20, 28, 17, 16, 23, 31, 19].		
Apply the multiplicative Holt-Winters method with m = 4, alpha = 0.5, beta = 0.3 and gamma = 0.2 to [30, 21, 29, 31, 40].	INVALID_INPUT	Holt-Winters multiplicative method: the series (two full seasons) needs at least 8 values.
Apply the multiplicative Holt-Winters method with alpha = 0.5, beta = 0.3 and gamma = 0.2 to [30, 21, 29, 31, 40, 30, 39, 41].	NEEDS_INPUT	holt_winters_multiplicative needs m (season length). Example: solve('holt_winters_multiplicative', x='30, 21, 29, 31, 40

Seasonal indices (additive decomposition) — `seasonal_indices_additive`

Seasonal indices of classical additive decomposition: detrend by the centred moving average, average each season, normalise the indices to sum to 0.

$$S_i = \bar{d}_i - \tfrac{1}{m}\sum_j \bar{d}_j, \quad d_t = x_t - \mathrm{CMA}_t$$

input	meaning	condition
`x`	the series, oldest first (at least two full seasons)	
`m`	season length	

output	meaning
`indices`	the m seasonal indices, starting with the season of x_1
`first`	the index of the first observation's season

- **Method:** differences x_t - CMA_t averaged by season, minus their mean
- **Assumes:** $n \geq 2m$; trend: centred moving average of length m (2 x m for even m); index i belongs to the season of observation i (the list starts at the first observation's season)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
time_series_ai999x.solve('seasonal_indices_additive', x='30, 21, 29, 31, 40, 30, 39, 41, 50, 39,
48, 51', m=4)
# -> indices = [6.484375, -6.453125, 0.171875, -0.203125], first = 6.48438   (SOLVED)
```

```
time_series_ai999x.solve('seasonal_indices_additive', x=[10, 14, 9, 11, 15, 10, 12, 16, 11], m=3)
# -> indices = [-0.6666666666666661, 3.0, -2.333333333333334], first = -0.666667   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the additive seasonal indices of the quarterly series [30, 21, 29, 31, 40, 30, 39, 41, 50, 39, 48, 51].	SOLVED	indices = [6.484375, -6.453125, 0.171875, -0.203125], first = 6.48438
Compute the additive seasonal indices with season length 3 of [10, 14, 9, 11, 15, 10, 12, 16, 11].	SOLVED	indices = [-0.6666666666666661, 3.0, -2.333333333333334], first = -0.666667

question	outcome	result
Find the additive seasonal indices with m = 4 of [1, 2, 3, 4, 5].	INVALID_INPUT	Seasonal indices (additive decomposition): the series (two full seasons) needs at least 8 values.
Find the additive seasonal indices of [1, 2, 3, 4, 5, 6, 7, 8].	NEEDS_INPUT	seasonal_indices_additive needs m (season length). Example: solve('seasonal_indices_additive', x='30, 21, 29, 31, 40, 30

Seasonally adjusted series (additive) — `seasonally_adjusted_additive`

The seasonally adjusted series of classical additive decomposition: each value minus its seasonal index.

$$x_t^{\mathrm{SA}} = x_t - S_t$$

input	meaning	condition
`x`	the series, oldest first (at least two full seasons)	
`m`	season length	

output	meaning
`adjusted`	the seasonally adjusted series
`last`	the latest adjusted value

- **Method:** seasonal indices of classical decomposition, then x_t - S_{season(t)}
- **Assumes:** $n \geq 2m$; trend: centred moving average of length m (2 x m for even m); index i belongs to the season of observation i (the list starts at the first observation's season)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
time_series_ai999x.solve('seasonally_adjusted_additive', x='30, 21, 29, 31, 40, 30, 39, 41, 50, 39,
48, 51', m=4)
# -> adjusted = [23.515625, 27.453125, 28.828125, 31.203125, 33.515625, 36.453125, 38.828125,
41.203125, 43.515625, 45.453125, 47.828125, 51.203125], last = 51.2031   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Seasonally adjust the quarterly series [30, 21, 29, 31, 40, 30, 39, 41, 50, 39, 48, 51] using additive decomposition.	SOLVED	adjusted = [23.515625, 27.453125, 28.828125, 31.203125, 33.515625, 36.453125, 38.828125, 41.203125, 43.515625, 45.453125, 47.828125, 51.203125], last = 51.2031
Seasonally adjust [1, 2, 3, 4, 5, 6] using additive decomposition.	NEEDS_INPUT	seasonally_adjusted_additive needs m (season length). Example: solve('seasonally_adjusted_additive', x='30, 21, 29, 31,

Seasonal indices (multiplicative decomposition) — `seasonal_indices_multiplicative`

Seasonal indices of classical multiplicative decomposition: detrend by the centred moving average, average each season, normalise the indices to average 1.

$$S_i = \bar{r}_i \Big/ \tfrac{1}{m}\sum_j \bar{r}_j, \quad r_t = x_t / \mathrm{CMA}_t$$

input	meaning	condition
`x`	the series, oldest first (at least two full seasons)	
`m`	season length	

output	meaning
`indices`	the m seasonal indices, starting with the season of x_1
`first`	the index of the first observation's season

- **Method:** ratios x_t / CMA_t averaged by season, divided by their mean
- **Assumes:** $n \geq 2m$; trend: centred moving average of length m (2 x m for even m); index i belongs to the season of observation i (the list starts at the first observation's season); positive data
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
time_series_ai999x.solve('seasonal_indices_multiplicative', x='30, 21, 29, 31, 40, 30, 39, 41, 50, 39, 48, 51', m=4)
# -> indices = [1.1675854591936505, 0.8390338031104935, 1.002044887955251, 0.9913358497406046], first = 1.16759   (SOLVED)
```

```
time_series_ai999x.solve('seasonal_indices_multiplicative', x=[10, 14, 9, 11, 15, 10, 12, 16, 11], m=3)
# -> indices = [0.9455091040549015, 1.2516901629290016, 0.8028007330160972], first = 0.945509   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the multiplicative seasonal indices of the quarterly series [30, 21, 29, 31, 40, 30, 39, 41, 50, 39, 48, 51].	SOLVED	indices = [1.1675854591936505, 0.8390338031104935, 1.002044887955251, 0.9913358497406046], first = 1.16759
Compute the multiplicative seasonal indices with season length 3 of [10, 14, 9, 11, 15, 10, 12, 16, 11].	SOLVED	indices = [0.9455091040549015, 1.2516901629290016, 0.8028007330160972], first = 0.945509
Find the multiplicative seasonal indices with m = 4 of [1, 2, 3, 4, 5].	INVALID_INPUT	Seasonal indices (multiplicative decomposition): the series (two full seasons) needs at least 8 values.
Find the multiplicative seasonal indices of [1, 2, 3, 4, 5, 6, 7, 8].	NEEDS_INPUT	seasonal_indices_multiplicative needs m (season length). Example: solve('seasonal_indices_multiplicative', x='30, 21, 29

Seasonally adjusted series (multiplicative) — `seasonally_adjusted_multiplicative`

The seasonally adjusted series of classical multiplicative decomposition: each value divided by its seasonal index.

$$x_t^{\mathrm{SA}} = x_t / S_t$$

input	meaning	condition
`x`	the series, oldest first (at least two full seasons)	
`m`	season length	

output	meaning
`adjusted`	the seasonally adjusted series
`last`	the latest adjusted value

- **Method:** seasonal indices of classical decomposition, then x_t / S_{season(t)}
- **Assumes:** $n \geq 2m$; trend: centred moving average of length m (2 x m for even m); index i belongs to the season of observation i (the list starts at the first observation's season); positive data
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
time_series_ai999x.solve('seasonally_adjusted_multiplicative', x='30, 21, 29, 31, 40, 30, 39, 41,
50, 39, 48, 51', m=4)
# -> adjusted = [25.694050712757576, 25.02878897387461, 28.940819267265272, 31.27093608902728,
34.25873428367677, 35.755412819820876, 38.9204121180464, 41.35833482742318, 42.82341785459596,
46.48203666576713, 47.902045683749414, 51.445733565819076], last = 51.4457   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Seasonally adjust the quarterly series [30, 21, 29, 31, 40, 30, 39, 41, 50, 39, 48, 51] using multiplicative decomposition.	SOLVED	adjusted = [25.694050712757576, 25.02878897387461, 28.940819267265272, 31.27093608902728, 34.25873428367677, 35.755412819820876, 38.9204121180464, 41.35833482742318, 42.82341785459596, 46.48203666576713, 47.902045683749414, 51.445733565819076], last = 51.4457
Seasonally adjust [1, 2, 3, 4, 5, 6] using multiplicative decomposition.	NEEDS_INPUT	seasonally_adjusted_multiplicative needs m (season length). Example: solve('seasonally_adjusted_multiplicative', x='30,

Regular and seasonal differencing — `differencing`

Difference a series d times (and seasonally D times at lag s): the series that an ARIMA(p, d, q) or seasonal ARIMA model works on.

$$\nabla^d \nabla_s^D x_t = (1 - L)^d (1 - L^s)^D x_t$$

input	meaning	condition
`x`	the series, oldest first	
`d`	number of regular (lag-1) differences, 0..3 (default 1)	
`D`	number of seasonal differences, 0..2 (default 0)	
`s`	seasonal lag (needed when D > 0)	

output	meaning
`differenced`	the differenced series
`last`	its latest value
`length`	its length n - d - D s

- **Method:** seasonal differences Missing superscript or subscript argument{t-s} (D times), then regular differences y_t - y_{$t - 1$} (d times)

- **Assumes:** $d + D \geq 1$; the series is longer than d + D s
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
time_series_ai999x.solve('differencing', x='3, 5, 9, 15, 23, 33', d=2)
# -> differenced = [2.0, 2.0, 2.0, 2.0], last = 2, length = 4   (SOLVED)
```

```
time_series_ai999x.solve('differencing', x=[10, 14, 9, 12, 11, 16, 10, 13], d=0, D=1, s=4)
# -> differenced = [1.0, 2.0, 1.0, 1.0], last = 1, length = 4   (SOLVED)
```

```
time_series_ai999x.solve('differencing', x='5, 8, 6, 9, 7, 10', d=1)
# -> differenced = [3.0, -2.0, 3.0, -2.0, 3.0], last = 3, length = 5   (SOLVED)
```

```
time_series_ai999x.solve('differencing', x=[10, 14, 9, 12, 11, 16, 10, 13, 12, 18, 11, 15], d=1,
D=1, s=4)
# -> differenced = [1.0, -1.0, 0.0, 0.0, 1.0, -1.0, 1.0], last = 1, length = 7   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the second differences of [3, 5, 9, 15, 23, 33].	SOLVED	differenced = [2.0, 2.0, 2.0, 2.0], last = 2, length = 4
Take the seasonal difference with lag 4 of [10, 14, 9, 12, 11, 16, 10, 13].	SOLVED	differenced = [1.0, 2.0, 1.0, 1.0], last = 1, length = 4
Compute the first differences of [5, 8, 6, 9, 7, 10].	SOLVED	differenced = [3.0, -2.0, 3.0, -2.0, 3.0], last = 3, length = 5
Apply a first difference and a seasonal difference with lag 4 to [10, 14, 9, 12, 11, 16, 10, 13, 12, 18, 11, 15].	SOLVED	differenced = [1.0, -1.0, 0.0, 0.0, 1.0, -1.0, 1.0], last = 1, length = 7
Take the seasonal difference with lag 12 of [1, 2, 3, 4, 5].	INVALID_INPUT	Regular and seasonal differencing: the series is too short for these differences.

Compound average growth rate of a series — `compound_average_growth_rate`

The compound (geometric) average growth rate per period of a positive series (CAGR when the periods are years), and the total growth from the first to the last value.

$$g = \left(\frac{x_n}{x_1}\right)^{1/(n-1)} - 1$$

input	meaning	condition
`x`	the series, oldest first (positive values)	

output	meaning
`rate`	compound average growth rate per period, as a fraction
`rate_percent`	the same rate in percent
`total_growth`	total growth $x_n/x_1 - 1$, as a fraction
`periods`	number of periods $n - 1$

- **Method:** (x_n / x_1)^(1/(n-1)) - 1
- **Assumes:** all values positive; equally spaced periods
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
time_series_ai999x.solve('compound_average_growth_rate', x='100, 110, 125, 133, 150')
# -> rate = 0.106682, rate_percent = 10.6682, total_growth = 0.5, periods = 4   (SOLVED)
```

```
time_series_ai999x.solve('compound_average_growth_rate', x=[2000, 2300, 2100, 2600])
# -> rate = 0.0913929, rate_percent = 9.13929, total_growth = 0.3, periods = 3   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the compound average growth rate of [100, 110, 125, 133, 150].	SOLVED	rate = 0.106682, rate_percent = 10.6682, total_growth = 0.5, periods = 4
What is the CAGR of the yearly revenues [2000, 2300, 2100, 2600]?	SOLVED	rate = 0.0913929, rate_percent = 9.13929, total_growth = 0.3, periods = 3
Find the compound average growth rate of [100, -5, 120].	INVALID_INPUT	Compound average growth rate of a series: the series must contain positive values only.

Sample autocorrelation at lag k — `sample_autocorrelation`

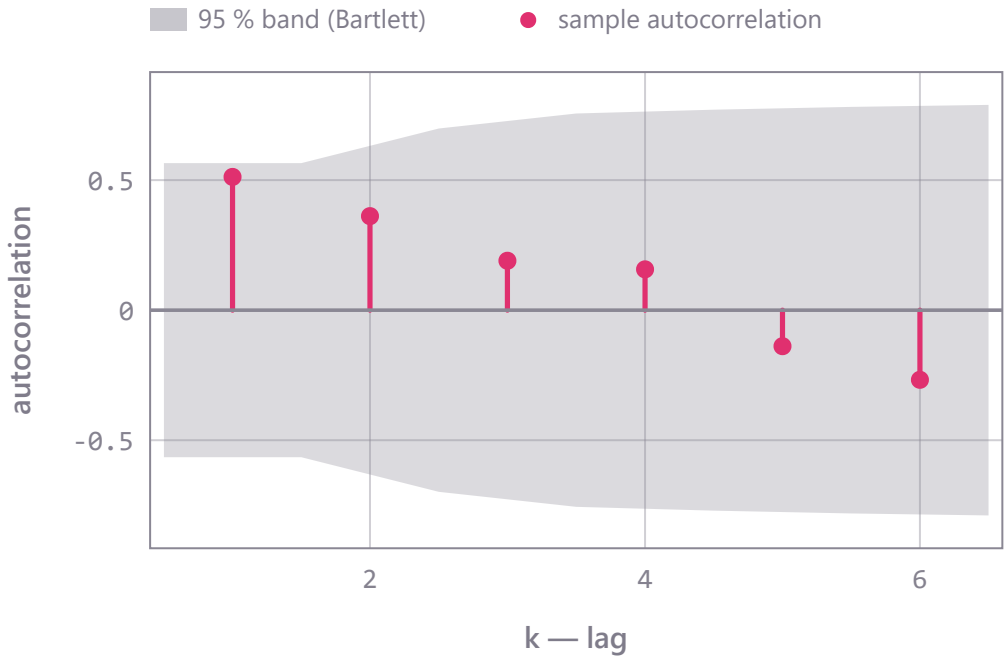
The sample autocorrelation r_k of a series (and r_1..r_k), with autocovariances divided by n and taken about the sample mean.

$$r_k = \frac{\sum_{t=1}^{n-k}(x_t - \bar{x})(x_{t+k} - \bar{x})}{\sum_{t=1}^{n}(x_t - \bar{x})^2}$$

input	meaning	condition
`x`	the series, oldest first	
`k`	the lag k $(1 \le k \le n-1)$	

output	meaning
`acf`	the sample autocorrelation r_k
`acf_list`	r_1, ..., r_k

- **Method:** $r_k = c_k/c_0$, $c_k = (1/n)$ sum_{$t = 1$}^{n-k} (x_t - xbar)(x_{t+k} - xbar)
- **Assumes:** the series is not constant; $1 \le k \le n-1$; denominator n (not n - k)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Sample autocorrelations of the series 30, 21, 29, 31, 40, 30, 39, 41, 50, 39, 48, 51 at lags 1 to 6 (0.512567, 0.361929, 0.190263, 0.157039, -0.138822, -0.267831) and the 95 % band ±0.565793 … ±0.789621 from Bartlett's standard errors, all returned by the package.

```
time_series_ai999x.solve('sample_autocorrelation', x='4, 6, 5, 8, 7, 9, 8, 10', k=1)
# -> acf = 0.272186, acf_list = [0.2721861471861472]   (SOLVED)
```

```
time_series_ai999x.solve('sample_autocorrelation', x=[12, 15, 11, 18, 14, 19, 13, 20, 16], k=3)
# -> acf = -0.429167, acf_list = [-0.4222222222222222, 0.5972222222222222, -0.42916666666666664]
(SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the lag 1 autocorrelation of [4, 6, 5, 8, 7, 9, 8, 10].	SOLVED	acf = 0.272186, acf_list = [0.2721861471861472]
Compute the sample autocorrelation at lag 3 of [12, 15, 11, 18, 14, 19, 13, 20, 16].	SOLVED	acf = -0.429167, acf_list = [-0.4222222222222222, 0.5972222222222222, -0.42916666666666664]
Find the lag 9 autocorrelation of [1, 2, 3].	INVALID_INPUT	Sample autocorrelation at lag k: the lag k must be at most 2.
Find the autocorrelation of [1, 2, 3, 5].	NEEDS_INPUT	sample_autocorrelation needs k (the lag k (1 <= k <= n - 1)). Example: solve('sample_autocorrelation', x='4, 6, 5, 8, 7,

Sample autocovariance at lag k — `sample_autocovariance`

The sample autocovariance c_k of a series, divided by n and taken about the sample mean.

$$c_k = \frac{1}{n}\sum_{t=1}^{n-k}(x_t - \bar{x})(x_{t+k} - \bar{x})$$

input	meaning	condition
`x`	the series, oldest first	

input	meaning	condition
`k`	the lag k $(0 \le k \le n-1)$	

output	meaning
`acov`	the sample autocovariance c_k
`gamma0`	c_0, the variance divided by n

- **Method:** $c_k = (1/n)$ sum_{$t = 1$}^{n-k} (x_t - xbar)(x_{t+k} - xbar)
- **Assumes:** $0 \le k \le n-1$; denominator n (not n - k)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
time_series_ai999x.solve('sample_autocovariance', x='4, 6, 5, 8, 7, 9, 8, 10', k=2)
# -> acov = 1.60547, gamma0 = 3.60938   (SOLVED)
```

```
time_series_ai999x.solve('sample_autocovariance', x=[1.5, 2.5, 2.0, 3.5, 3.0], k=1)
# -> acov = 0, gamma0 = 0.5   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the lag 2 autocovariance of [4, 6, 5, 8, 7, 9, 8, 10].	SOLVED	acov = 1.60547, gamma0 = 3.60938
Compute the sample autocovariance at lag 1 of [1.5, 2.5, 2.0, 3.5, 3.0].	SOLVED	acov = 0, gamma0 = 0.5
Find the autocovariance at lag 7 of [1, 2, 3].	INVALID_INPUT	Sample autocovariance at lag k: the lag k must be at most 2.

Sample partial autocorrelation at lag k — `sample_partial_autocorrelation`

The sample partial autocorrelation phi_kk of a series (and phi_11..phi_kk) by the Durbin-Levinson recursion on the sample autocorrelations (denominator n).

$$\phi_{kk} = \frac{r_k - \sum_{j=1}^{k-1} \phi_{k-1,j}\, r_{k-j}}{1 - \sum_{j=1}^{k-1} \phi_{k-1,j}\, r_j}$$

input	meaning	condition
`x`	the series, oldest first	
`k`	the lag k $(1 \le k < n/2)$	

output	meaning
`pacf`	the partial autocorrelation at lag k
`pacf_list`	partial autocorrelations at lags 1..k

- **Method:** Durbin-Levinson: phi_kk = (r_k - sum phi_{$k - 1$,j} r_{k-j}) / (1 - sum phi_{$k - 1$,j} r_j)
- **Assumes:** the series is not constant; $1 \le k < n/2$; autocovariances divided by n
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
time_series_ai999x.solve('sample_partial_autocorrelation', x='4, 6, 5, 8, 7, 9, 8, 10, 9, 12', k=2)
# -> pacf = 0.401658, pacf_list = [0.3596899224806201, 0.40165819531144314]   (SOLVED)
```

```
time_series_ai999x.solve('sample_partial_autocorrelation', x=[3.1, 2.4, 3.9, 3.3, 4.6, 3.8, 5.0,
4.1, 5.6, 4.9, 6.2, 5.1], k=3)
# -> pacf = -0.380129, pacf_list = [0.4228595178719867, 0.40003524472063134, -0.38012917986902794]
(SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the partial autocorrelation at lag 2 of [4, 6, 5, 8, 7, 9, 8, 10, 9, 12].	SOLVED	pacf = 0.401658, pacf_list = [0.3596899224806201, 0.40165819531144314]
Compute the PACF at lag 3 of [3.1, 2.4, 3.9, 3.3, 4.6, 3.8, 5.0, 4.1, 5.6, 4.9, 6.2, 5.1].	SOLVED	pacf = -0.380129, pacf_list = [0.4228595178719867, 0.40003524472063134, -0.38012917986902794]
Find the partial autocorrelation at lag 3 of [1, 2, 4, 3, 5].	INVALID_INPUT	Sample partial autocorrelation at lag k: the lag k must be less than n/2.

Bartlett standard error of a sample autocorrelation — `bartlett_standard_error`

Bartlett's standard error of r_k under the hypothesis that the autocorrelations beyond lag $k-1$ are zero, and the 95 % band half-width used in correlograms.

$$\mathrm{se}(r_k) = \sqrt{\frac{1 + 2\sum_{j=1}^{k-1} r_j^2}{n}}$$

input	meaning	condition
`x`	the series, oldest first	
`k`	the lag k	

output	meaning
`se`	Bartlett standard error of r_k
`half_width_95`	1.959964 x se
`acf`	the sample autocorrelation r_k

- **Method:** $se_k = sqrt$((1 + 2 sum_{$j < k$} r_j^2) / n)
- **Assumes:** $1 \le k \le n-1$; autocorrelations with denominator n
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
time_series_ai999x.solve('bartlett_standard_error', x='4, 6, 5, 8, 7, 9, 8, 10, 9, 12', k=3)
# -> se = 0.41446, half_width_95 = 0.812326, acf = -0.0410853   (SOLVED)
```

```
time_series_ai999x.solve('bartlett_standard_error', x=[12, 15, 11, 18, 14, 19, 13, 20, 16], k=1)
# -> se = 0.333333, half_width_95 = 0.653321, acf = -0.422222   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the Bartlett standard error of the lag 3 autocorrelation of [4, 6, 5, 8, 7, 9, 8, 10, 9, 12].	SOLVED	se = 0.41446, half_width_95 = 0.812326, acf = -0.0410853
What is the standard error of the sample autocorrelation at lag 1 of [12, 15, 11, 18, 14, 19, 13, 20, 16]?	SOLVED	se = 0.333333, half_width_95 = 0.653321, acf = -0.422222

Ljung-Box test — `ljung_box_test`

The Ljung-Box test portmanteau statistic for no autocorrelation up to lag h and its chi-square p-value (h - model_df degrees of freedom; autocorrelations with denominator n).

$$Q_{LB} = n(n+2)\sum_{k=1}^{h}\frac{r_k^2}{n-k}$$

input	meaning	condition
`x`	the series (or residuals), oldest first	
`h`	the number of lags h	
`model_df`	parameters of a fitted ARMA model subtracted from the degrees of freedom (default 0)	

output	meaning
`q`	the test statistic
`p_value`	upper-tail chi-square probability, df = h - model_df
`df`	degrees of freedom h - model_df

- **Method:** $Q = n$(n+2) sum_{$k = 1$}^h $r_k^2/(n-k)$; $p = P$(chi2_{h-model_df} > Q)
- **Assumes:** $1 \le h \le n-1$; model_df < h (default 0: raw series)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
time_series_ai999x.solve('ljung_box_test', x='4, 6, 5, 8, 7, 9, 8, 10, 9, 12, 11, 13', h=3)
# -> q = 9.3191, p_value = 0.0253358, df = 3   (SOLVED)
```

```
time_series_ai999x.solve('ljung_box_test', x=[0.5, -0.3, 0.8, -0.1, 0.2, -0.6, 0.4, 0.1, -0.2, 0.7,
-0.4, 0.3, 0.0, -0.5, 0.6], h=5)
# -> q = 10.4225, p_value = 0.0641114, df = 5   (SOLVED)
```

```
time_series_ai999x.solve('ljung_box_test', x=[0.5, -0.3, 0.8, -0.1, 0.2, -0.6, 0.4, 0.1, -0.2, 0.7,
-0.4, 0.3, 0.0, -0.5, 0.6], h=6, model_df=2)
# -> q = 11.6382, p_value = 0.0202543, df = 4   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Perform the Ljung-Box test with 3 lags on [4, 6, 5, 8, 7, 9, 8, 10, 9, 12, 11, 13].	SOLVED	q = 9.3191, p_value = 0.0253358, df = 3
Compute the Ljung-Box test statistic with h = 5 for the residuals [0.5, -0.3, 0.8, -0.1, 0.2, -0.6, 0.4, 0.1, -0.2, 0.7,	SOLVED	q = 10.4225, p_value = 0.0641114, df = 5

question	outcome	result
-0.4, 0.3, 0.0, -0.5, 0.6].		
Perform the Ljung-Box test with 6 lags and 2 fitted parameters on the residuals [0.5, -0.3, 0.8, -0.1, 0.2, -0.6, 0.4, 0.1, -0.2, 0.7, -0.4, 0.3, 0.0, -0.5, 0.6].	SOLVED	q = 11.6382, p_value = 0.0202543, df = 4
Perform the Ljung-Box test on [1, 3, 2, 4, 3].	NEEDS_INPUT	ljung_box_test needs h (the number of lags h). Example: solve('ljung_box_test', x='4, 6, 5, 8, 7, 9, 8, 10, 9, 12, 11, 1

Box-Pierce test — `box_pierce_test`

The Box-Pierce test portmanteau statistic for no autocorrelation up to lag h and its chi-square p-value (h - model_df degrees of freedom; autocorrelations with denominator n).

$$Q_{BP} = n \sum_{k=1}^{h} r_k^2$$

input	meaning	condition
`x`	the series (or residuals), oldest first	
`h`	the number of lags h	
`model_df`	parameters of a fitted ARMA model subtracted from the degrees of freedom (default 0)	

output	meaning
`q`	the test statistic
`p_value`	upper-tail chi-square probability, df = h - model_df
`df`	degrees of freedom h - model_df

- **Method:** $Q = n$ sum_{$k = 1$}^h r_k^2; $p = P$(chi2_{h-model_df} > Q)
- **Assumes:** $1 \leq h \leq n - 1$; model_df < h (default 0: raw series)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
time_series_ai999x.solve('box_pierce_test', x='4, 6, 5, 8, 7, 9, 8, 10, 9, 12, 11, 13', h=3)
# -> q = 6.9439, p_value = 0.0737077, df = 3   (SOLVED)
```

```
time_series_ai999x.solve('box_pierce_test', x=[0.5, -0.3, 0.8, -0.1, 0.2, -0.6, 0.4, 0.1, -0.2,
0.7, -0.4, 0.3, 0.0, -0.5, 0.6], h=5)
# -> q = 8.0139, p_value = 0.155471, df = 5   (SOLVED)
```

```
time_series_ai999x.solve('box_pierce_test', x=[0.5, -0.3, 0.8, -0.1, 0.2, -0.6, 0.4, 0.1, -0.2,
0.7, -0.4, 0.3, 0.0, -0.5, 0.6], h=6, model_df=2)
# -> q = 8.65753, p_value = 0.0702536, df = 4   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Perform the Box-Pierce test with 3 lags on [4, 6, 5, 8, 7, 9, 8, 10, 9, 12, 11, 13].	SOLVED	q = 6.9439, p_value = 0.0737077, df = 3
Compute the Box-Pierce test statistic with h = 5 for the residuals [0.5, -0.3, 0.8, -0.1, 0.2, -0.6, 0.4, 0.1, -0.2, 0.7, -0.4, 0.3, 0.0, -0.5, 0.6].	SOLVED	q = 8.0139, p_value = 0.155471, df = 5
Perform the Box-Pierce test with 6 lags and 2 fitted parameters on the residuals [0.5, -0.3, 0.8, -0.1, 0.2, -0.6, 0.4, 0.1, -0.2, 0.7, -0.4, 0.3, 0.0, -0.5, 0.6].	SOLVED	q = 8.65753, p_value = 0.0702536, df = 4
Perform the Box-Pierce test on [1, 3, 2, 4, 3].	NEEDS_INPUT	box_pierce_test needs h (the number of lags h). Example: solve('box_pierce_test', x='4, 6, 5, 8, 7, 9, 8, 10, 9, 12, 11,

AR(p) estimation by Yule-Walker — `ar_yule_walker`

Estimate the coefficients phi_1..phi_p and the innovation variance of an AR(p) model from data by the Yule-Walker equations (sample autocovariances divided by n, mean removed).

$$\begin{pmatrix} 1 & r_1 & \cdots \\ r_1 & 1 & \cdots \\ \vdots & & \ddots \end{pmatrix} \phi = \begin{pmatrix} r_1 \\ \vdots \\ r_p \end{pmatrix}$$

input	meaning	condition
`x`	the series, oldest first	
`p`	the AR order p	

output	meaning
`phi`	the coefficients phi_1..phi_p
`phi1`	phi_1
`sigma2`	the innovation variance c_0 (1 - sum phi_j r_j)

- **Method:** solve the Toeplitz system R $\phi = r$ (Durbin-Levinson); $\sigma^2 = c_0$ prod(1 - phi_kk^2)
- **Assumes:** the series is not constant; $1 \le p < n - 1$; denominator n
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
time_series_ai999x.solve('ar_yule_walker', x='4, 6, 5, 8, 7, 9, 8, 10, 9, 12', p=2)
# -> phi = [0.21521751734534134, 0.40165819531144314], phi1 = 0.215218, sigma2 = 3.76766   (SOLVED)
```

```
time_series_ai999x.solve('ar_yule_walker', x=[1.2, 0.8, 1.5, 1.1, 1.9, 1.4, 2.0, 1.6], p=1)
# -> phi = [-0.0111141602634467], phi1 = -0.0111142, sigma2 = 0.142326   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Estimate an AR(2) model by Yule-Walker for the series [4, 6, 5, 8, 7, 9, 8, 10, 9, 12].	SOLVED	phi = [0.21521751734534134, 0.40165819531144314], phi1 = 0.215218, sigma2 = 3.76766

question	outcome	result
Fit an AR(1) model with the Yule-Walker equations to the data [1.2, 0.8, 1.5, 1.1, 1.9, 1.4, 2.0, 1.6].	SOLVED	phi = [-0.0111141602634467], phi1 = -0.0111142, sigma2 = 0.142326
Fit an autoregressive model by Yule-Walker to the data [1, 3, 2, 4].	NEEDS_INPUT	ar_yule_walker needs p (the AR order p). Example: solve('ar_yule_walker', x='4, 6, 5, 8, 7, 9, 8, 10, 9, 12', p=2).

AR(p) estimation by conditional least squares — `ar_least_squares`

Fit $x_t = c + phi_1$ x_{$t-1$} + ... + phi_p x_{t-p} + e_t by ordinary least squares on $t = p+1..n$; the innovation variance is the residual sum of squares divided by n - p.

$$\min_{c,\phi} \sum_{t=p+1}^{n} \left(x_t - c - \sum_{j=1}^{p} \phi_j x_{t-j} \right)^2$$

input	meaning	condition
`x`	the series, oldest first	
`p`	the AR order p	

output	meaning
`const`	the constant c
`phi`	the coefficients phi_1..phi_p
`phi1`	phi_1
`sigma2`	residual sum of squares / (n - p)

- **Method:** least squares (Householder QR) of x_t on (1, x_{$t-1$}, ..., x_{t-p}), $t = p+1..n$
- **Assumes:** $n - p > p + 2$; regressors not collinear; the first p values are conditioned on
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
time_series_ai999x.solve('ar_least_squares', x='4, 6, 5, 8, 7, 9, 8, 10, 9, 12', p=1)
# -> const = 3.5625, phi = [0.6354166666666665], phi1 = 0.635417, sigma2 = 2.51505   (SOLVED)
```

```
time_series_ai999x.solve('ar_least_squares', x=[3.1, 2.4, 3.9, 3.3, 4.6, 3.8, 5.0, 4.1, 5.6, 4.9,
6.2, 5.1], p=2)
# -> const = 1.23321, phi = [-0.08029317028950547, 0.925914839174237], phi1 = -0.0802932, sigma2 =
0.0304673   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Fit an AR(1) model by least squares to [4, 6, 5, 8, 7, 9, 8, 10, 9, 12].	SOLVED	const = 3.5625, phi = [0.6354166666666665], phi1 = 0.635417, sigma2 = 2.51505
Estimate an AR(2) by OLS for [3.1, 2.4, 3.9, 3.3, 4.6, 3.8, 5.0, 4.1, 5.6, 4.9, 6.2, 5.1].	SOLVED	const = 1.23321, phi = [-0.08029317028950547, 0.925914839174237], phi1 = -0.0802932, sigma2 = 0.0304673
Fit an AR(3) model by least squares to [1, 2, 3, 2, 1].	INVALID_INPUT	AR(p) estimation by conditional least squares: too few observations: need n - p > p + 2.

Yule-Walker equations from autocorrelations — `yule_walker_from_autocorrelations`

Solve the Yule-Walker equations for the AR(p) coefficients given the autocorrelations rho_1..rho_p, and the ratio of the innovation variance to the process variance.

$$\rho_k = \sum_{j=1}^{p} \phi_j \, \rho_{|k-j|}, \; k = 1..p$$

input	meaning	condition
`rho`	the autocorrelations rho_1..rho_p	

output	meaning
`phi`	the AR coefficients phi_1..phi_p
`phi1`	phi_1
`variance_ratio`	sigma^2 / gamma_0 = 1 - sum phi_j rho_j

- **Method:** Durbin-Levinson recursion on rho_1..rho_p
- **Assumes:** the autocorrelations form a positive definite sequence
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
time_series_ai999x.solve('yule_walker_from_autocorrelations', rho='0.6, 0.2')
# -> phi = [0.75, -0.24999999999999994], phi1 = 0.75, variance_ratio = 0.6   (SOLVED)
```

```
time_series_ai999x.solve('yule_walker_from_autocorrelations', rho=[0.8, 0.5, 0.3])
# -> phi = [1.1818181818181828, -0.5909090909090924, 0.1818181818181826], phi1 = 1.18182,
variance_ratio = 0.295455   (SOLVED)
```

```
time_series_ai999x.solve('yule_walker_from_autocorrelations', rho='0.5')
# -> phi = [0.5], phi1 = 0.5, variance_ratio = 0.75   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Solve the Yule-Walker equations for the autocorrelations [0.6, 0.2].	SOLVED	phi = [0.75, -0.24999999999999994], phi1 = 0.75, variance_ratio = 0.6
Find the AR(3) coefficients from the Yule-Walker equations with autocorrelations [0.8, 0.5, 0.3].	SOLVED	phi = [1.1818181818181828, -0.5909090909090924, 0.1818181818181826], phi1 = 1.18182, variance_ratio = 0.295455
Solve the Yule-Walker equations for the autocorrelations [0.9, 0.1].	INVALID_INPUT	Yule-Walker equations from autocorrelations: the autocorrelations are not a valid (positive definite) sequence.
Find the AR(2) coefficients from the Yule-Walker equations with autocorrelations [0.5].	OUT_OF_SCOPE	Out of scope: AR(2) was named but 1 coefficients were given.

Partial autocorrelations from autocorrelations — `partial_autocorrelations_from_autocorrelations`

The partial autocorrelations phi_11..phi_pp implied by autocorrelations rho_1..rho_p (Durbin-Levinson).

$$\phi_{kk} = \frac{\rho_k - \sum_{j=1}^{k-1} \phi_{k-1,j}\rho_{k-j}}{1 - \sum_{j=1}^{k-1} \phi_{k-1,j}\rho_j}$$

input	meaning	condition
`rho`	the autocorrelations rho_1..rho_p	

output	meaning
`pacf`	partial autocorrelations at lags 1..p
`last`	phi_pp

- **Method:** Durbin-Levinson recursion
- **Assumes:** the autocorrelations form a positive definite sequence
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
time_series_ai999x.solve('partial_autocorrelations_from_autocorrelations', rho='0.6, 0.3, 0.1')
# -> pacf = [0.6, -0.09375, -0.06403940886699507], last = -0.0640394   (SOLVED)
```

```
time_series_ai999x.solve('partial_autocorrelations_from_autocorrelations', rho=[0.8, 0.64])
# -> pacf = [0.8, -3.0839528461809913e-16], last = -3.08395e-16   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the partial autocorrelations from the autocorrelations [0.6, 0.3, 0.1].	SOLVED	pacf = [0.6, -0.09375, -0.06403940886699507], last = -0.0640394
Compute the PACF given autocorrelations [0.8, 0.64].	SOLVED	pacf = [0.8, -3.0839528461809913e-16], last = -3.08395e-16
Find the partial autocorrelations from the autocorrelations [0.9, 0.1].	INVALID_INPUT	Partial autocorrelations from autocorrelations: the autocorrelations are not a valid (positive definite) sequence.

Forecasts of an AR(p) model — `ar_forecast`

h-step forecasts of $x_t = c + phi_1$ x_{$t - 1$} + ... + phi_p x_{t-p} + e_t from the last p observations (future shocks set to 0).

$$\hat{x}_{n+j} = c + \sum_{i=1}^{p} \phi_i \, \hat{x}_{n+j-i}$$

input	meaning	condition
`x`	the series, oldest first (at least p values)	
`phi`	the coefficients phi_1..phi_p	
`c`	the constant c (default 0)	
`h`	forecast horizon (default 1)	

output	meaning
`forecasts`	the forecasts for steps 1..h
`forecast`	the h-step forecast

- **Method:** recursion xhat_{n+j} = c + sum_i phi_i xhat_{n+j-i} (observed values where available)
- **Assumes:** phi_1 multiplies the latest observation
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
time_series_ai999x.solve('ar_forecast', x='10, 11, 12', phi='0.5, 0.2', c=1, h=3)
# -> forecasts = [9.2, 8.0, 6.84], forecast = 6.84   (SOLVED)
```

```
time_series_ai999x.solve('ar_forecast', x=[4.0, 5.5], phi=[0.8], h=2)
# -> forecasts = [4.4, 3.5200000000000005], forecast = 3.52   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Forecast 3 steps ahead with the AR(2) model with coefficients [0.5, 0.2] and constant 1 from the series [10, 11, 12].	SOLVED	forecasts = [9.2, 8.0, 6.84], forecast = 6.84
Forecast 2 steps ahead for the series [4.0, 5.5] with the AR(1) model with coefficients [0.8].	SOLVED	forecasts = [4.4, 3.5200000000000005], forecast = 3.52
Forecast 3 steps ahead with the AR(3) model with coefficients [0.5, 0.2] from the series [10, 11, 12].	OUT_OF_SCOPE	Out of scope: AR(3) was named but 2 coefficients were given.

Forecasts of an ARIMA(p,1,0) model — `arima_p10_forecast`

h-step forecasts of an ARIMA(p,1,0) model without constant: the differences follow a stationary AR(p); forecast differences are accumulated onto the last observation.

$$(1 - \phi_1 L - \cdots - \phi_p L^p)(1 - L)x_t = e_t$$

input	meaning	condition
`x`	the series, oldest first (at least $p + 2$ values)	
`phi`	AR coefficients of the differences	
`h`	forecast horizon (default 1)	

output	meaning
`forecasts`	the forecasts for steps 1..h
`forecast`	the h-step forecast

- **Method:** forecast Missing superscript or subscript argument{$t - 1$} by the AR(p) recursion, xhat_{n+j} = xhat_{$n + j - 1$} + dhat_{n+j}
- **Assumes:** stationary AR part; no constant (no drift); $n \geq p + 2$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
time_series_ai999x.solve('arima_p10_forecast', x='100, 102, 105, 104, 108', phi='0.4', h=3)
# -> forecasts = [109.6, 110.24, 110.496], forecast = 110.496   (SOLVED)
```

```
time_series_ai999x.solve('arima_p10_forecast', x=[50, 51, 53, 52, 55, 57], phi=[0.5, -0.2], h=2)
# -> forecasts = [57.4, 57.199999999999996], forecast = 57.2   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Forecast 3 steps ahead with an ARIMA(1,1,0) model with coefficients [0.4] for the series [100, 102, 105, 104, 108].	SOLVED	forecasts = [109.6, 110.24, 110.496], forecast = 110.496
Forecast 2 steps ahead with an ARIMA(2,1,0) model with coefficients [0.5, -0.2] for the series [50, 51, 53, 52, 55, 57].	SOLVED	forecasts = [57.4, 57.199999999999996], forecast = 57.2
Forecast with an ARIMA(1,1,0) model with coefficients [1.2] for the series [1, 2, 4, 7].	INVALID_INPUT	Forecasts of an ARIMA(p,1,0) model: the AR part must be stationary (every root outside the unit circle).

Theoretical autocorrelations of an MA(q) process — `ma_theoretical_acf`

The autocorrelations rho_1..rho_K of $x_t = e_t + theta_1$ e_{$t-1$} + ... + theta_q e_{t-q}; they vanish beyond lag q.

$$\rho_k = \frac{\theta_k + \sum_{j=1}^{q-k} \theta_j \theta_{j+k}}{1 + \sum_{j=1}^{q} \theta_j^2}$$

input	meaning	condition
`theta`	the MA coefficients theta_1..theta_q (plus-sign convention)	
`K`	number of lags (default q + 1)	

output	meaning
`acf`	rho_1..rho_K
`rho1`	rho_1

- **Method:** rho_k = (theta_k + sum_{$j=1$}^{q-k} theta_j theta_{j+k}) / (1 + sum theta_j^2), 0 for $k > q$
- **Assumes:** x_t = e_t + sum theta_j e_{t-j} (plus signs)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
time_series_ai999x.solve('ma_theoretical_acf', theta='0.5, 0.3')
# -> acf = [0.48507462686567165, 0.22388059701492535, 0.0], rho1 = 0.485075   (SOLVED)
```

```
time_series_ai999x.solve('ma_theoretical_acf', theta=[0.8], K=3)
# -> acf = [0.4878048780487805, 0.0, 0.0], rho1 = 0.487805   (SOLVED)
```

```
time_series_ai999x.solve('ma_theoretical_acf', theta='-0.4, 0.2, 0.6', K=4)
# -> acf = [-0.23076923076923078, -0.025641025641025626, 0.3846153846153846, 0.0], rho1 = -0.230769
(SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the autocorrelations of the MA(2) process with theta = [0.5, 0.3].	SOLVED	acf = [0.48507462686567165, 0.22388059701492535, 0.0], rho1 = 0.485075
Compute the theoretical ACF up to lag 3 of the MA(1) model with theta = [0.8].	SOLVED	acf = [0.4878048780487805, 0.0, 0.0], rho1 = 0.487805
Find the autocorrelations up to lag 4 of the MA(3) process with theta = [-0.4, 0.2, 0.6].	SOLVED	acf = [-0.23076923076923078, -0.025641025641025626, 0.3846153846153846, 0.0], rho1 = -0.230769
Find the autocorrelations of the MA(2) process with theta = [0.5].	OUT_OF_SCOPE	Out of scope: MA(2) was named but 1 coefficients were given.

Theoretical autocorrelations of an ARMA(1,1) process — `arma11_theoretical_acf`

The autocorrelations rho_1..rho_K of $x_t = \phi$ x_{$t-1$} + e_t + theta e_{$t-1$}.

$$\rho_1 = \frac{(1+\phi\theta)(\phi+\theta)}{1+2\phi\theta+\theta^2},\ \rho_k = \phi\,\rho_{k-1}$$

input	meaning	condition
`phi`	AR coefficient, |phi| < 1	
`theta`	MA coefficient (plus-sign convention)	
`K`	number of lags (default 5)	

output	meaning
`acf`	rho_1..rho_K
`rho1`	rho_1

- **Method:** rho_1 = (1 + phi theta)(phi + theta)/(1 + 2 phi theta + theta^2); $rho_k = \phi$ rho_{$k-1$}
- **Assumes:** \|phi\| < 1; $x_t = \phi$ x_{$t-1$} + e_t + theta e_{$t-1$}
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
time_series_ai999x.solve('arma11_theoretical_acf', phi=0.6, theta=0.3)
# -> acf = [0.7324137931034482, 0.4394482758620689, 0.26366896551724134, 0.1582013793103448,
0.09492082758620689], rho1 = 0.732414   (SOLVED)
```

```
time_series_ai999x.solve('arma11_theoretical_acf', phi=-0.5, theta=0.8, K=3)
# -> acf = [0.2142857142857143, -0.10714285714285715, 0.053571428571428575], rho1 = 0.214286
(SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the autocorrelations of the ARMA(1,1) process with phi = 0.6 and theta = 0.3.	SOLVED	acf = [0.7324137931034482, 0.4394482758620689, 0.26366896551724134, 0.1582013793103448, 0.09492082758620689], rho1 = 0.732414
Compute the theoretical ACF up to lag 3 of the ARMA(1,1) model with phi = -0.5 and theta = 0.8.	SOLVED	acf = [0.2142857142857143, -0.10714285714285715, 0.053571428571428575], rho1 = 0.214286
Find the autocorrelations of the ARMA(1,1) process with phi = 1.2 and theta = 0.3.	INVALID_INPUT	Theoretical autocorrelations of an ARMA(1,1) process: |phi| must be less than 1 (stationarity).

Theoretical autocorrelations of an ARMA(p,q) process — `arma_theoretical_acf`

The autocorrelations rho_1..rho_K of a stationary ARMA(p,q) process x_t = sum phi_i x_{t-i} + e_t + sum theta_j e_{t-j}.

$$\gamma_k - \sum_{i=1}^{p} \phi_i \gamma_{k-i} = \sigma^2 \sum_{j=k}^{q} \theta_j \psi_{j-k}$$

input	meaning	condition
`phi`	AR coefficients phi_1..phi_p (stationary; [0] for none)	
`theta`	MA coefficients theta_1..theta_q ([0] for none)	
`K`	number of lags (default 5)	

output	meaning
`acf`	rho_1..rho_K
`rho1`	rho_1

- **Method:** solve the linear equations for gamma_0..gamma_p, then gamma_k = sum phi_i gamma_{k-i} + σ^2 sum theta_j psi_{j-k}
- **Assumes:** stationary AR part; plus-sign MA convention
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
time_series_ai999x.solve('arma_theoretical_acf', phi='0.5, 0.2', theta='0.4')
# -> acf = [0.8012048192771084, 0.6006024096385542, 0.46054216867469877, 0.35039156626506024,
0.2673042168674699], rho1 = 0.801205   (SOLVED)
```

```
time_series_ai999x.solve('arma_theoretical_acf', phi=[0.7], theta=[0.2, -0.3], K=4)
# -> acf = [0.665406976744186, 0.31752906976744183, 0.22227034883720928, 0.15558924418604647], rho1
= 0.665407   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the autocorrelations of the ARMA(2,1) process with phi = [0.5, 0.2] and theta = [0.4].	SOLVED	acf = [0.8012048192771084, 0.6006024096385542, 0.46054216867469877, 0.35039156626506024, 0.2673042168674699], rho1 = 0.801205

question	outcome	result
Compute the theoretical ACF up to lag 4 of the ARMA(1,2) model with phi = [0.7] and theta = [0.2, -0.3].	SOLVED	acf = [0.665406976744186, 0.31752906976744183, 0.22227034883720928, 0.15558924418604647], rho1 = 0.665407
Find the autocorrelations of the ARMA(1,1) process with phi = [1.1] and theta = [0.4].	INVALID_INPUT	Theoretical autocorrelations of an ARMA(p,q) process: the AR part must be stationary (every root outside the unit circle

Variance and lag-1 autocovariance of an ARMA(p,q) process — `arma_variance`

The process variance gamma_0 and lag-1 autocovariance gamma_1 of a stationary ARMA(p,q) process with innovation variance σ^2.

$$\gamma_0 = \sigma^2 \sum_{j \geq 0} \psi_j^2$$

input	meaning	condition
`phi`	AR coefficients ([0] for none)	
`theta`	MA coefficients ([0] for none)	
`sigma2`	innovation (white-noise) variance σ^2	

output	meaning
`gamma0`	the variance of the process
`gamma1`	the lag-1 autocovariance

- **Method:** exact linear equations for the autocovariances
- **Assumes:** stationary AR part; $\sigma^2 > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
time_series_ai999x.solve('arma_variance', phi='0.5', theta='0.4', sigma2=2)
# -> gamma0 = 4.16, gamma1 = 2.88   (SOLVED)
```

```
time_series_ai999x.solve('arma_variance', phi=[0.6, -0.2], theta=[0.3], sigma2=1)
# -> gamma0 = 1.93056, gamma1 = 1.21528   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the variance of the ARMA(1,1) process with phi = [0.5], theta = [0.4] and sigma2 = 2.	SOLVED	gamma0 = 4.16, gamma1 = 2.88
Compute the autocovariance of the ARMA(2,1) model with phi = [0.6, -0.2], theta = [0.3] and white noise variance 1.	SOLVED	gamma0 = 1.93056, gamma1 = 1.21528
Find the variance of the ARMA(1,1) process with phi = [0.5] and theta = [0.4].	NEEDS_INPUT	arma_variance needs sigma2 (innovation (white-noise) variance sigma^2). Example: solve('arma_variance', phi='0.5', theta

Psi weights (MA-infinity representation) of an ARMA process — `psi_weights`

The weights psi_1..psi_K of x_t = sum_{$j \geq 0$} psi_j e_{t-j} ($psi_0 = 1$) for a stationary ARMA(p,q); they are also the impulse response.

$$\psi_j = \theta_j + \sum_{i=1}^{\min(j,p)} \phi_i \, \psi_{j-i}, \; \psi_0 = 1$$

input	meaning	condition
`phi`	AR coefficients ([0] for none)	
`theta`	MA coefficients ([0] for none)	
`K`	number of weights (default 10)	

output	meaning
`psi`	psi_1..psi_K
`psi1`	$psi_1 = phi_1 + theta_1$

- **Method:** psi_j = theta_j + sum_{$i = 1$}^{min(j,p)} phi_i psi_{j-i}
- **Assumes:** stationary AR part; plus-sign MA convention
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
time_series_ai999x.solve('psi_weights', phi='0.5', theta='0.3', K=5)
# -> psi = [0.8, 0.4, 0.2, 0.1, 0.05], psi1 = 0.8   (SOLVED)
```

```
time_series_ai999x.solve('psi_weights', phi=[0.6, -0.2], theta=[0.4, 0.1], K=6)
# -> psi = [1.0, 0.5, 0.09999999999999998, -0.04000000000000002, -0.04400000000000001, -0.0184],
psi1 = 1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the first 5 psi weights of the ARMA(1,1) process with phi = [0.5] and theta = [0.3].	SOLVED	psi = [0.8, 0.4, 0.2, 0.1, 0.05], psi1 = 0.8
Compute the first 6 psi weights of the ARMA(2,2) model with phi = [0.6, -0.2] and theta = [0.4, 0.1].	SOLVED	psi = [1.0, 0.5, 0.09999999999999998, -0.04000000000000002, -0.04400000000000001, -0.0184], psi1 = 1

Pi weights (AR-infinity representation) of an ARMA process — `pi_weights`

The weights pi_1..pi_K of x_t = sum_{$j \geq 1$} pi_j x_{t-j} + e_t for an invertible ARMA(p,q).

$$\pi(L) = 1 - \sum_{j \geq 1} \pi_j L^j = \frac{\phi(L)}{\theta(L)}$$

input	meaning	condition
`phi`	AR coefficients ([0] for none)	
`theta`	MA coefficients ([0] for none), invertible	
`K`	number of weights (default 10)	

output	meaning
`pi`	pi_1..pi_K
`pi1`	$pi_1 = phi_1 + theta_1$

- **Method:** coefficients c_j of phi(L)/theta(L); $pi_j = -c_j$
- **Assumes:** invertible MA part; stationary AR part; plus-sign MA convention
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
time_series_ai999x.solve('pi_weights', phi='0.5', theta='0.3', K=5)
# -> pi = [0.8, -0.24, 0.072, -0.021599999999999998, 0.006479999999999999], pi1 = 0.8   (SOLVED)
```

```
time_series_ai999x.solve('pi_weights', phi=[0], theta=[0.6], K=4)
# -> pi = [0.6, -0.36, 0.216, -0.1296], pi1 = 0.6   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the first 5 pi weights of the ARMA(1,1) process with phi = [0.5] and theta = [0.3].	SOLVED	pi = [0.8, -0.24, 0.072, -0.021599999999999998, 0.006479999999999999], pi1 = 0.8
Compute the first 4 pi weights of the MA(1) model with phi = [0] and theta = [0.6].	SOLVED	pi = [0.6, -0.36, 0.216, -0.1296], pi1 = 0.6
Find the first 4 pi weights of the ARMA(1,1) process with phi = [0.5] and theta = [1.5].	INVALID_INPUT	Pi weights (AR-infinity representation) of an ARMA process: the MA part must be invertible (every root outside the unit

h-step forecast error variance of an ARMA model — `forecast_error_variance`

The variance (and standard deviation) of the h-step-ahead forecast error of a stationary ARMA(p,q) with known parameters: σ^2 (psi_0^2 + ... + psi_{$h-1$}^2).

$$\mathrm{Var}(e_n(h)) = \sigma^2 \sum_{j=0}^{h-1} \psi_j^2$$

input	meaning	condition
`phi`	AR coefficients ([0] for none)	
`theta`	MA coefficients ([0] for none)	
`sigma2`	innovation variance σ^2	
`h`	forecast horizon	

output	meaning
`variance`	forecast error variance
`sd`	forecast error standard deviation

- **Method:** psi weights, then σ^2 sum_{$j = 0$}^{$h - 1$} psi_j^2
- **Assumes:** stationary AR part; known parameters (no estimation error)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
time_series_ai999x.solve('forecast_error_variance', phi='0.5', theta='0.3', sigma2=2, h=3)
# -> variance = 3.6, sd = 1.89737   (SOLVED)
```

```
time_series_ai999x.solve('forecast_error_variance', phi=[0.8], theta=[0], sigma2=1, h=5)
# -> variance = 2.47952, sd = 1.57465   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the 3-step forecast error variance of the ARMA(1,1) model with phi = [0.5], theta = [0.3] and sigma2 = 2.	SOLVED	variance = 3.6, sd = 1.89737
What is the variance of the 5-step-ahead forecast error of the ARMA(1,0) model with phi = [0.8], theta = [0] and sigma2 = 1?	SOLVED	variance = 2.47952, sd = 1.57465

Mean, variance and autocorrelation of an AR(1) process — `ar1_properties`

For a stationary AR(1) $x_t = c + \phi$ x_{$t - 1$} + e_t: the mean $c/(1 - \phi)$, the variance $\sigma^2/(1 - \phi^2)$, and the lag-k autocorrelation ϕ^k and autocovariance.

$$\mu = \frac{c}{1 - \phi},\ \gamma_0 = \frac{\sigma^2}{1 - \phi^2},\ \rho_k = \phi^k$$

input	meaning	condition
`phi`	AR coefficient, |phi| < 1	
`sigma2`	innovation variance σ^2	
`k`	the lag k (default 1)	
`c`	the constant c (default 0)	

output	meaning
`mean`	$c/(1 - \phi)$
`variance`	$\sigma^2/(1 - \phi^2)$
`acf_k`	ϕ^k
`acov_k`	variance x ϕ^k

- **Method:** stationary moments of the AR(1) recursion
- **Assumes:** \|phi\| < 1; $\sigma^2 > 0$

- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
time_series_ai999x.solve('ar1_properties', phi=0.6, sigma2=4, c=2)
# -> mean = 5, variance = 6.25, acf_k = 0.6, acov_k = 3.75   (SOLVED)
```

```
time_series_ai999x.solve('ar1_properties', phi=-0.5, sigma2=1, k=3)
# -> mean = 0, variance = 1.33333, acf_k = -0.125, acov_k = -0.166667   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the mean and variance of the AR(1) process with constant 2, phi = 0.6 and sigma2 = 4.	SOLVED	mean = 5, variance = 6.25, acf_k = 0.6, acov_k = 3.75
Find the lag 3 autocorrelation and variance of the AR(1) process with phi = -0.5 and white noise variance 1.	SOLVED	mean = 0, variance = 1.33333, acf_k = -0.125, acov_k = -0.166667
Find the variance of the AR(1) process with phi = 1.1 and sigma2 = 1.	INVALID_INPUT	Mean, variance and autocorrelation of an AR(1) process: |phi| must be less than 1 (stationarity).
Find the variance of the AR(1) process with phi = 0.5.	NEEDS_INPUT	ar1_properties needs sigma2 (innovation variance sigma^2). Example: solve('ar1_properties', phi=0.6, sigma2=4, c=2).

Stationarity of an AR(2) process — `ar2_stationarity`

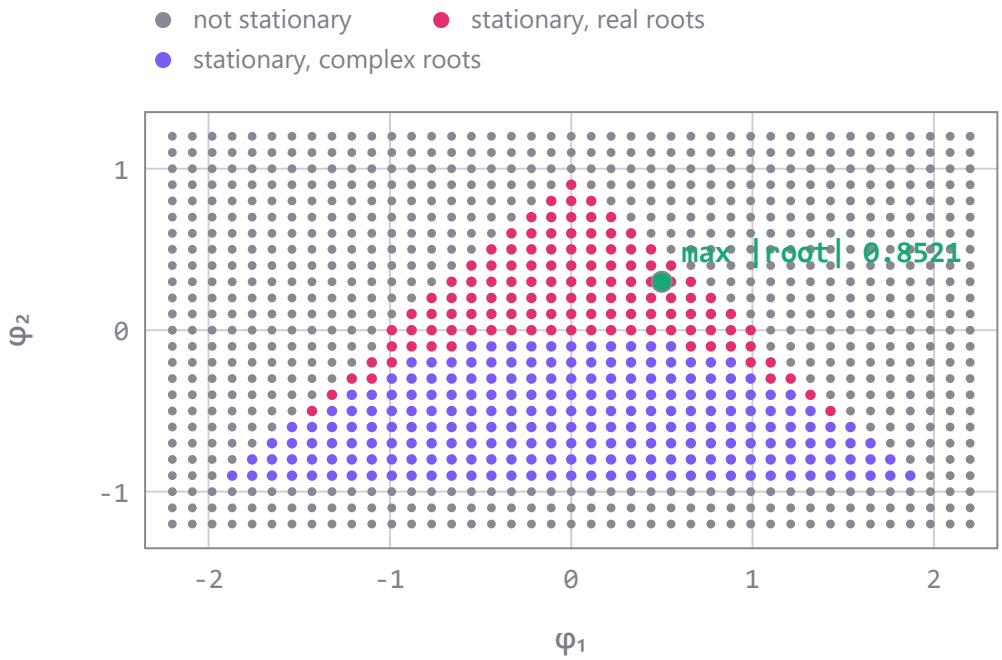
Whether $x_t = phi1$ x_{$t - 1$} + phi2 x_{$t - 2$} + e_t is stationary: the largest modulus of the inverse characteristic roots (roots of $z^2 - phi1$ z - phi2) must be below 1.

$$z^2 - \phi_1 z - \phi_2 = 0, \quad \text{stationary} \iff \max |z_i| < 1$$

input	meaning	condition
`phi1`	phi_1	
`phi2`	phi_2	

output	meaning
`stationary`	yes or no
`max_modulus`	largest modulus of the inverse roots
`roots`	real or complex

- **Method:** roots of $z^2 - phi1\ z - phi2$ by the quadratic formula
- **Assumes:** a modulus of exactly 1 (unit root) counts as not stationary
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Which AR(2) models x_t = φ₁x_(t−1) + φ₂x_(t−2) + e_t are stationary: the package's verdict at 1025 points of the (φ₁, φ₂) plane — stationary with real roots (120 points), stationary with complex roots, i.e. damped oscillation (223), or not stationary (682); together they trace the stationarity triangle. Marked: φ₁ = 0.5, φ₂ = 0.3, real roots, largest modulus 0.85208.

```
time_series_ai999x.solve('ar2_stationarity', phi1=0.5, phi2=0.3)
# -> stationary = yes, max_modulus = 0.85208, roots = real   (SOLVED)
```

```
time_series_ai999x.solve('ar2_stationarity', phi1=1.2, phi2=-0.8)
# -> stationary = yes, max_modulus = 0.894427, roots = complex   (SOLVED)
```

```
time_series_ai999x.solve('ar2_stationarity', phi1=0.6, phi2=0.5)
# -> stationary = no, max_modulus = 1.06811, roots = real   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Is the AR(2) process with phi1 = 0.5 and phi2 = 0.3 stationary?	SOLVED	stationary = yes, max_modulus = 0.85208, roots = real
Check the stationarity of the AR(2) model with phi1 = 1.2 and phi2 = -0.8.	SOLVED	stationary = yes, max_modulus = 0.894427, roots = complex
Is the AR(2) process with phi1 = 0.6 and phi2 = 0.5 stationary?	SOLVED	stationary = no, max_modulus = 1.06811, roots = real
Is the AR(2) process with phi1 = 0.5 stationary?	NEEDS_INPUT	ar2_stationarity needs phi2 (phi_2). Example: solve('ar2_stationarity', phi1=0.5, phi2=0.3).

Invertibility of an MA(2) process — `ma2_invertibility`

Whether $x_t = e_t + theta1$ e_{$t - 1$} + theta2 e_{$t - 2$} is invertible: the largest modulus of the inverse roots (roots of $z^2 + theta1$ z + theta2) must be below 1.

$$z^2 + \theta_1 z + \theta_2 = 0, \quad \text{invertible} \iff \max |z_i| < 1$$

input	meaning	condition
`theta1`	theta_1	
`theta2`	theta_2	

output	meaning
`invertible`	yes or no
`max_modulus`	largest modulus of the inverse roots
`roots`	real or complex

- **Method:** roots of $z^2 + theta1\ z + theta2$ by the quadratic formula
- **Assumes:** plus-sign MA convention; a modulus of exactly 1 counts as not invertible
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
time_series_ai999x.solve('ma2_invertibility', theta1=0.5, theta2=0.3)
# -> invertible = yes, max_modulus = 0.547723, roots = complex   (SOLVED)
```

```
time_series_ai999x.solve('ma2_invertibility', theta1=-1.5, theta2=0.9)
# -> invertible = yes, max_modulus = 0.948683, roots = complex   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Is the MA(2) process with theta1 = 0.5 and theta2 = 0.3 invertible?	SOLVED	invertible = yes, max_modulus = 0.547723, roots = complex
Check the invertibility of the MA(2) model with theta1 = -1.5 and theta2 = 0.9.	SOLVED	invertible = yes, max_modulus = 0.948683, roots = complex
Is the MA(2) process with theta1 = 0.5 invertible?	NEEDS_INPUT	ma2_invertibility needs theta2 (theta_2). Example: solve('ma2_invertibility', theta1=0.5, theta2=0.3).

Sample cross-correlation at lag k — `cross_correlation_at_lag`

The sample cross-correlation r_xy(k) = corr(x_{t+k}, y_t) of two equally long series (y leads x by k).

$$r_{xy}(k) = \frac{\frac{1}{n}\sum_{t=1}^{n-k}(x_{t+k} - \bar{x})(y_t - \bar{y})}{s_x s_y}$$

input	meaning	condition
`x`	the first series	
`y`	the second series (same length)	
`k`	the lag $k \geq 0$	

output	meaning
`ccf`	the cross-correlation r_xy(k)

- **Method:** r_xy(k) = (1/n) sum_t (x_{t+k} - xbar)(y_t - ybar) / (s_x s_y)
- **Assumes:** equal lengths; $0 \leq k \leq n - 1$; denominator n, standard deviations with divisor n

- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
time_series_ai999x.solve('cross_correlation_at_lag', x='1, 3, 2, 5, 4, 6', y='2, 1, 4, 3, 6, 5',
k=1)
# -> ccf = 0.642857   (SOLVED)
```

```
time_series_ai999x.solve('cross_correlation_at_lag', x=[10, 12, 11, 15, 14, 18, 17], y=[8, 11, 10,
12, 16, 15, 19], k=2)
# -> ccf = 0.24262   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Compute the cross-correlation at lag 1 between x = [1, 3, 2, 5, 4, 6] and y = [2, 1, 4, 3, 6, 5].	SOLVED	ccf = 0.642857
Find the sample cross-correlation at lag 2 of x = [10, 12, 11, 15, 14, 18, 17] and y = [8, 11, 10, 12, 16, 15, 19].	SOLVED	ccf = 0.24262
Compute the cross-correlation at lag 1 between x = [1, 3, 2] and y = [2, 1, 4, 3].	INVALID_INPUT	Sample cross-correlation at lag k: x and y must have the same length.

Lag of the strongest cross-correlation — `cross_correlation_best_lag`

The lag k in -K..K with the largest |r_xy(k)|, r_xy(k) = corr(x_{t+k}, y_t): a positive k means y leads x by k periods.

$$k^* = \arg\max_{|k| \le K} |r_{xy}(k)|$$

input	meaning	condition
`x`	the first series	
`y`	the second series (same length)	
`K`	the largest lag searched	

output	meaning
`lag`	the lag with the largest absolute cross-correlation
`ccf`	r_xy at that lag

- **Method:** r_xy(k) for every k in -K..K, the largest in absolute value
- **Assumes:** equal lengths; $K \le n - 1$; ties: the smallest |k|, then the positive k
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
time_series_ai999x.solve('cross_correlation_best_lag', x='1, 3, 2, 5, 4, 6, 5, 8', y='3, 2, 5, 4,
6, 5, 8, 7', K=3)
# -> lag = 1, ccf = 0.713657   (SOLVED)
```

```
time_series_ai999x.solve('cross_correlation_best_lag', x=[10, 12, 11, 15, 14, 18, 17, 20], y=[12,
11, 15, 14, 18, 17, 20, 19], K=2)
# -> lag = 0, ccf = 0.764246   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
At which lag up to 3 is the cross-correlation strongest between x = [1, 3, 2, 5, 4, 6, 5, 8] and y = [3, 2, 5, 4, 6, 5, 8, 7]?	SOLVED	lag = 1, ccf = 0.713657
Find the lag with the largest cross-correlation, max lag 2, of x = [10, 12, 11, 15, 14, 18, 17, 20] and y = [12, 11, 15, 14, 18, 17, 20, 19].	SOLVED	lag = 0, ccf = 0.764246

Mean absolute error — `mean_absolute_error`

The mean absolute error of forecasts against actual values.

$$\mathrm{MAE} = \frac{1}{n}\sum_{t=1}^{n}|y_t - f_t|$$

input	meaning	condition
`y`	the actual values	
`f`	the forecasts (same length)	

output	meaning
`mae`	Mean absolute error
`n`	number of forecasts

- **Method:** mean of |actual - forecast|
- **Assumes:** equal lengths; errors e_t = actual - forecast
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
time_series_ai999x.solve('mean_absolute_error', y='10, 12, 14, 13', f='11, 12, 13, 15')
# -> mae = 1, n = 4   (SOLVED)
```

```
time_series_ai999x.solve('mean_absolute_error', y=[102.5, 98.0, 110.2, 105.7, 99.9], f=[100.0,
101.0, 107.5, 104.0, 103.2])
# -> mae = 2.64, n = 5   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the MAE for actual values [10, 12, 14, 13] and forecasts [11, 12, 13, 15].	SOLVED	mae = 1, n = 4
Compute the mean absolute error for actual values [102.5, 98.0, 110.2, 105.7, 99.9] and forecasts [100.0, 101.0, 107.5, 104.0, 103.2].	SOLVED	mae = 2.64, n = 5

question	outcome	result
Find the MAE for actual values [1, 2, 3] and forecasts [1, 2].	INVALID_INPUT	Mean absolute error: the actual values and the forecasts must have the same length.
Find the MAE for forecasts [1, 2, 3].	NEEDS_INPUT	mean_absolute_error needs y (the actual values). Example: solve('mean_absolute_error', y='10, 12, 14, 13', f='11, 12, 13

Mean squared error — `mean_squared_error`

The mean squared error of forecasts against actual values.

$$\mathrm{MSE} = \frac{1}{n}\sum_{t=1}^{n}(y_t - f_t)^2$$

input	meaning	condition
`y`	the actual values	
`f`	the forecasts (same length)	

output	meaning
`mse`	Mean squared error
`n`	number of forecasts

- **Method:** mean of (actual - forecast)^2
- **Assumes:** equal lengths; errors e_t = actual - forecast
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
time_series_ai999x.solve('mean_squared_error', y='10, 12, 14, 13', f='11, 12, 13, 15')
# -> mse = 1.5, n = 4   (SOLVED)
```

```
time_series_ai999x.solve('mean_squared_error', y=[102.5, 98.0, 110.2, 105.7, 99.9], f=[100.0,
101.0, 107.5, 104.0, 103.2])
# -> mse = 7.264, n = 5   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the MSE for actual values [10, 12, 14, 13] and forecasts [11, 12, 13, 15].	SOLVED	mse = 1.5, n = 4
Compute the mean squared error for actual values [102.5, 98.0, 110.2, 105.7, 99.9] and forecasts [100.0, 101.0, 107.5, 104.0, 103.2].	SOLVED	mse = 7.264, n = 5
Find the MSE for actual values [1, 2, 3] and forecasts [1, 2].	INVALID_INPUT	Mean squared error: the actual values and the forecasts must have the same length.
Find the MSE for forecasts [1, 2, 3].	NEEDS_INPUT	mean_squared_error needs y (the actual values). Example: solve('mean_squared_error', y='10, 12, 14, 13', f='11, 12, 13,

Root mean squared error — `root_mean_squared_error`

The root mean squared error of forecasts against actual values.

$$\mathrm{RMSE} = \sqrt{\frac{1}{n}\sum_{t=1}^{n}(y_t - f_t)^2}$$

input	meaning	condition
`y`	the actual values	
`f`	the forecasts (same length)	

output	meaning
`rmse`	Root mean squared error
`n`	number of forecasts

- **Method:** square root of the mean squared error
- **Assumes:** equal lengths; errors e_t = actual - forecast
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
time_series_ai999x.solve('root_mean_squared_error', y='10, 12, 14, 13', f='11, 12, 13, 15')
# -> rmse = 1.22474, n = 4   (SOLVED)
```

```
time_series_ai999x.solve('root_mean_squared_error', y=[102.5, 98.0, 110.2, 105.7, 99.9], f=[100.0,
101.0, 107.5, 104.0, 103.2])
# -> rmse = 2.69518, n = 5   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the RMSE for actual values [10, 12, 14, 13] and forecasts [11, 12, 13, 15].	SOLVED	rmse = 1.22474, n = 4
Compute the root mean squared error for actual values [102.5, 98.0, 110.2, 105.7, 99.9] and forecasts [100.0, 101.0, 107.5, 104.0, 103.2].	SOLVED	rmse = 2.69518, n = 5
Find the RMSE for actual values [1, 2, 3] and forecasts [1, 2].	INVALID_INPUT	Root mean squared error: the actual values and the forecasts must have the same length.
Find the RMSE for forecasts [1, 2, 3].	NEEDS_INPUT	root_mean_squared_error needs y (the actual values). Example: solve('root_mean_squared_error', y='10, 12, 14, 13', f='11

Mean absolute percentage error — `mean_absolute_percentage_error`

The mean absolute percentage error (in percent) of forecasts against actual values.

$$\mathrm{MAPE} = \frac{100}{n}\sum_{t=1}^{n}\left|\frac{y_t - f_t}{y_t}\right|$$

input	meaning	condition
`y`	the actual values	
`f`	the forecasts (same length)	

output	meaning
`mape`	Mean absolute percentage error
`n`	number of forecasts

- **Method:** 100 x mean of |actual - forecast| / |actual|
- **Assumes:** equal lengths; errors e_t = actual - forecast; no actual value is 0; the result is in percent
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
time_series_ai999x.solve('mean_absolute_percentage_error', y='10, 12, 14, 13', f='11, 12, 13, 15')
# -> mape = 8.13187, n = 4   (SOLVED)
```

```
time_series_ai999x.solve('mean_absolute_percentage_error', y=[102.5, 98.0, 110.2, 105.7, 99.9], f=
[100.0, 101.0, 107.5, 104.0, 103.2])
# -> mape = 2.57239, n = 5   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the MAPE for actual values [10, 12, 14, 13] and forecasts [11, 12, 13, 15].	SOLVED	mape = 8.13187, n = 4
Compute the mean absolute percentage error for actual values [102.5, 98.0, 110.2, 105.7, 99.9] and forecasts [100.0, 101.0, 107.5, 104.0, 103.2].	SOLVED	mape = 2.57239, n = 5
Find the MAPE for actual values [1, 2, 3] and forecasts [1, 2].	INVALID_INPUT	Mean absolute percentage error: the actual values and the forecasts must have the same length.
Find the MAPE for forecasts [1, 2, 3].	NEEDS_INPUT	mean_absolute_percentage_error needs y (the actual values). Example: solve('mean_absolute_percentage_error', y='10, 12,

Mean forecast error (bias) — `mean_forecast_error`

The mean forecast error (bias): positive when the forecasts are too low on average.

$$\mathrm{ME} = \frac{1}{n}\sum_{t=1}^{n}(y_t - f_t)$$

input	meaning	condition
`y`	the actual values	

input	meaning	condition
`f`	the forecasts (same length)	

output	meaning
`mean_error`	Mean forecast error (bias)
`n`	number of forecasts

- **Method:** mean of actual - forecast (computed by the reference as mean(actual) - mean(forecast))
- **Assumes:** equal lengths; errors e_t = actual - forecast
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
time_series_ai999x.solve('mean_forecast_error', y='10, 12, 14, 13', f='11, 12, 13, 15')
# -> mean_error = -0.5, n = 4   (SOLVED)
```

```
time_series_ai999x.solve('mean_forecast_error', y=[102.5, 98.0, 110.2, 105.7, 99.9], f=[100.0,
101.0, 107.5, 104.0, 103.2])
# -> mean_error = 0.12, n = 5   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the mean forecast error for actual values [10, 12, 14, 13] and forecasts [11, 12, 13, 15].	SOLVED	mean_error = -0.5, n = 4
Compute the mean forecast error (bias) for actual values [102.5, 98.0, 110.2, 105.7, 99.9] and forecasts [100.0, 101.0, 107.5, 104.0, 103.2].	SOLVED	mean_error = 0.12, n = 5
Find the mean forecast error for actual values [1, 2, 3] and forecasts [1, 2].	INVALID_INPUT	Mean forecast error (bias): the actual values and the forecasts must have the same length.
Find the mean forecast error for forecasts [1, 2, 3].	NEEDS_INPUT	mean_forecast_error needs y (the actual values). Example: solve('mean_forecast_error', y='10, 12, 14, 13', f='11, 12, 13

Symmetric mean absolute percentage error — `symmetric_mape`

The symmetric MAPE in percent: $100/n$ sum 2|y - f| / (|y| + |f|).

$$\mathrm{sMAPE} = \frac{100}{n} \sum_{t=1}^{n} \frac{2|y_t - f_t|}{|y_t| + |f_t|}$$

input	meaning	condition
`y`	the actual values	
`f`	the forecasts (same length)	

output	meaning
`smape`	Symmetric mean absolute percentage error
`n`	number of forecasts

- **Method:** 100 x mean of 2|actual - forecast| / (|actual| + |forecast|)
- **Assumes:** equal lengths; errors e_t = actual - forecast; no pair with actual and forecast both 0; the result is in percent (range 0 to 200)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
time_series_ai999x.solve('symmetric_mape', y='10, 12, 14, 13', f='11, 12, 13, 15')
# -> smape = 7.80423, n = 4   (SOLVED)
```

```
time_series_ai999x.solve('symmetric_mape', y=[102.5, 98.0, 110.2, 105.7, 99.9], f=[100.0, 101.0,
107.5, 104.0, 103.2])
# -> smape = 2.56714, n = 5   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the sMAPE for actual values [10, 12, 14, 13] and forecasts [11, 12, 13, 15].	SOLVED	smape = 7.80423, n = 4
Compute the symmetric mean absolute percentage error for actual values [102.5, 98.0, 110.2, 105.7, 99.9] and forecasts [100.0, 101.0, 107.5, 104.0, 103.2].	SOLVED	smape = 2.56714, n = 5
Find the sMAPE for actual values [1, 2, 3] and forecasts [1, 2].	INVALID_INPUT	Symmetric mean absolute percentage error: the actual values and the forecasts must have the same length.
Find the sMAPE for forecasts [1, 2, 3].	NEEDS_INPUT	symmetric_mape needs y (the actual values). Example: solve('symmetric_mape', y='10, 12, 14, 13', f='11, 12, 13, 15').

Mean absolute scaled error — `mean_absolute_scaled_error`

MASE: the forecasts' mean absolute error divided by the in-sample mean absolute error of the (seasonal) naive forecast on the training data.

$$\mathrm{MASE} = \frac{\frac{1}{h}\sum |y_t - f_t|}{\frac{1}{T-m}\sum_{t=m+1}^{T} |y_t - y_{t-m}|}$$

input	meaning	condition
`y_train`	the training (in-sample) data	
`y`	the actual values	
`f`	the forecasts (same length as y)	
`sp`	seasonal period of the naive benchmark (default 1)	

output	meaning
`mase`	the mean absolute scaled error
`mae`	the forecasts' MAE
`scale`	in-sample MAE of the lag-sp naive forecast

- **Method:** MAE(actual, forecast) / mean |y_train_t - y_train_{t-sp}|
- **Assumes:** the training data are longer than sp and not constant at lag sp
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
time_series_ai999x.solve('mean_absolute_scaled_error', y_train='10, 12, 11, 14, 13', y='15, 14, 16', f='14, 15, 15')
# -> mase = 0.571429, mae = 1, scale = 1.75   (SOLVED)
```

```
time_series_ai999x.solve('mean_absolute_scaled_error', y_train=[20, 30, 25, 22, 31, 27], y=[24, 33], f=[23, 30], sp=3)
# -> mase = 1.2, mae = 2, scale = 1.66667   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the MASE for training data [10, 12, 11, 14, 13], actual values [15, 14, 16] and forecasts [14, 15, 15].	SOLVED	mase = 0.571429, mae = 1, scale = 1.75
Compute the mean absolute scaled error with seasonal period 3 for training data [20, 30, 25, 22, 31, 27], actual values [24, 33] and forecasts [23, 30].	SOLVED	mase = 1.2, mae = 2, scale = 1.66667
Find the MASE for actual values [15, 14, 16] and forecasts [14, 15, 15].	NEEDS_INPUT	mean_absolute_scaled_error needs y_train (the training (in-sample) data). Example: solve('mean_absolute_scaled_error', y

Linear trend fit and forecast — `linear_trend_forecast`

Fit the least-squares trend line $x_t = a + b\,t$ $(t = 1..n)$ and forecast period n + h.

$$\hat{x}_t = a + b\,t, \quad \hat{x}_{n+h} = a + b(n+h)$$

input	meaning	condition
`x`	the series, oldest first	
`h`	periods ahead to forecast (default 1)	

output	meaning
`slope`	trend per period b
`intercept`	a (the trend value at t = 0)
`forecast`	a + b (n + h)
`r_squared`	coefficient of determination

- **Method:** ordinary least squares of x_t on (1, t)
- **Assumes:** $t = 1..n$; the series is not constant
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
time_series_ai999x.solve('linear_trend_forecast', x='12, 15, 17, 20, 22, 25', h=2)
# -> slope = 2.54286, intercept = 9.6, forecast = 29.9429, r_squared = 0.996979   (SOLVED)
```

```
time_series_ai999x.solve('linear_trend_forecast', x=[5.2, 5.0, 5.9, 6.3, 6.1, 7.0, 7.4])
# -> slope = 0.385714, intercept = 4.58571, forecast = 7.67143, r_squared = 0.906716   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Fit a linear trend to [12, 15, 17, 20, 22, 25] and forecast 2 periods ahead.	SOLVED	slope = 2.54286, intercept = 9.6, forecast = 29.9429, r_squared = 0.996979
Find the least-squares trend line of [5.2, 5.0, 5.9, 6.3, 6.1, 7.0, 7.4].	SOLVED	slope = 0.385714, intercept = 4.58571, forecast = 7.67143, r_squared = 0.906716
Fit a linear trend to [4, 4, 4, 4].	INVALID_INPUT	Linear trend fit and forecast: the series must not be constant.

Quadratic trend fit and forecast — `quadratic_trend_forecast`

Fit $x_t = b_0 + b_1\,t + b_2\,t^2\ (t = 1..n)$ by least squares and forecast period n + h.

$$\hat{x}_t = b_0 + b_1 t + b_2 t^2$$

input	meaning	condition
`x`	the series, oldest first	
`h`	periods ahead (default 1)	

output	meaning
`b0`	constant
`b1`	coefficient of t
`b2`	coefficient of t^2
`forecast`	the trend value at $t = n + h$

- **Method:** least squares (QR) on (1, t, t^2)
- **Assumes:** $t = 1..n$; $n \geq 4$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
time_series_ai999x.solve('quadratic_trend_forecast', x='3, 5, 9, 15, 22, 32', h=1)
# -> b0 = 2.8, b1 = -0.728571, b2 = 0.928571, forecast = 43.2   (SOLVED)
```

```
time_series_ai999x.solve('quadratic_trend_forecast', x=[100, 98, 99, 103, 110, 120, 133], h=3)
# -> b0 = 105, b1 = -6.5, b2 = 1.5, forecast = 190   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Fit a quadratic trend to [3, 5, 9, 15, 22, 32] and forecast 1 period ahead.	SOLVED	b0 = 2.8, b1 = -0.728571, b2 = 0.928571, forecast = 43.2
Fit a quadratic trend to [100, 98, 99, 103, 110, 120, 133] and forecast 3 periods ahead.	SOLVED	b0 = 105, b1 = -6.5, b2 = 1.5, forecast = 190

Exponential trend fit and forecast — `exponential_trend_forecast`

Fit $x_t = a$ e^(b t) $(t = 1..n)$ by least squares on ln x_t, the growth rate per period $e^b - 1$, and the forecast of period n + h.

$$\ln x_t = \ln a + b\,t, \quad \hat{x}_{n+h} = a\,e^{b(n+h)}$$

input	meaning	condition
`x`	the series, oldest first (positive)	
`h`	periods ahead (default 1)	

output	meaning
`a`	the scale a
`b`	the exponent rate b
`growth_rate`	$e^b - 1$ per period
`forecast`	a e^(b (n + h))

- **Method:** ordinary least squares of ln x_t on (1, t)
- **Assumes:** positive values; least squares on the logarithms (not on the values)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
time_series_ai999x.solve('exponential_trend_forecast', x='100, 112, 124, 140, 155', h=1)
# -> a = 89.6495, b = 0.109965, growth_rate = 0.116239, forecast = 173.417   (SOLVED)
```

```
time_series_ai999x.solve('exponential_trend_forecast', x=[2.0, 2.3, 2.9, 3.1, 3.8, 4.4], h=2)
# -> a = 1.71304, b = 0.157579, growth_rate = 0.170673, forecast = 6.043   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Fit an exponential trend to [100, 112, 124, 140, 155] and forecast 1 period ahead.	SOLVED	a = 89.6495, b = 0.109965, growth_rate = 0.116239, forecast = 173.417
Fit an exponential trend to [2.0, 2.3, 2.9, 3.1, 3.8, 4.4] and forecast 2 periods ahead.	SOLVED	a = 1.71304, b = 0.157579, growth_rate = 0.170673, forecast = 6.043
Fit an exponential trend to [5, -1, 3].	INVALID_INPUT	Exponential trend fit and forecast: the series must contain positive values only.

Sen's slope (Theil-Sen trend) — `sens_slope`

Sen's robust trend slope (median of all pairwise slopes, t = 1..n) and the intercept median(x) - slope median(t).

$$\hat{\beta} = \mathrm{median}_{i<j} \frac{x_j - x_i}{j - i}$$

input	meaning	condition
`x`	the series, oldest first	

output	meaning
`slope`	median of (x_j - x_i)/(j - i), $i < j$
`intercept`	median(x) - slope $(n+1)/2$

- **Method:** median of the n(n-1)/2 pairwise slopes
- **Assumes:** $t = 1..n$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
time_series_ai999x.solve('sens_slope', x='12, 14, 13, 17, 16, 19, 25')
# -> slope = 1.66667, intercept = 9.33333   (SOLVED)
```

```
time_series_ai999x.solve('sens_slope', x=[3.2, 3.0, 3.9, 4.1, 3.8, 4.6])
# -> slope = 0.266667, intercept = 2.91667   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find Sen's slope of [12, 14, 13, 17, 16, 19, 25].	SOLVED	slope = 1.66667, intercept = 9.33333
Compute the Theil-Sen trend of [3.2, 3.0, 3.9, 4.1, 3.8, 4.6].	SOLVED	slope = 0.266667, intercept = 2.91667

Mann-Kendall trend test — `mann_kendall_test`

The Mann-Kendall statistic S, its variance (with the tie correction), the continuity-corrected normal score z and the two-sided p-value for a monotonic trend.

$$S = \sum_{i<j} \mathrm{sgn}(x_j - x_i), \;\; \mathrm{Var}\, S = \frac{n(n-1)(2n+5) - \sum t(t-1)(2t+5)}{18}$$

input	meaning	condition
`x`	the series, oldest first	

output	meaning
`s`	S = sum_{$i < j$} sign(x_j - x_i)
`var_s`	variance of S with tie correction
`z`	(S - sign S) / sqrt(var S)
`p_value`	two-sided normal p-value

- **Method:** count concordant minus discordant pairs; normal approximation
- **Assumes:** $n \geq 4$; ties corrected in the variance; continuity correction of 1 in z

- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
time_series_ai999x.solve('mann_kendall_test', x='12, 14, 13, 17, 16, 19, 25, 21, 24')
# -> s = 28, var_s = 92, z = 2.81494, p_value = 0.00487856   (SOLVED)
```

```
time_series_ai999x.solve('mann_kendall_test', x=[5, 3, 5, 6, 4, 7, 7, 8, 6, 9])
# -> s = 28, var_s = 122, z = 2.44447, p_value = 0.0145067   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Perform the Mann-Kendall trend test on [12, 14, 13, 17, 16, 19, 25, 21, 24].	SOLVED	s = 28, var_s = 92, z = 2.81494, p_value = 0.00487856
Compute the Mann-Kendall statistic for [5, 3, 5, 6, 4, 7, 7, 8, 6, 9].	SOLVED	s = 28, var_s = 122, z = 2.44447, p_value = 0.0145067
Perform the Mann-Kendall trend test on [3, 3, 3, 3, 3].	INVALID_INPUT	Mann-Kendall trend test: the series must not be constant.

Linear interpolation of a missing value — `linear_interpolation`

Fill a missing value at time t0 by the straight line between the neighbouring observations.

$$\hat{y}(t_0) = y_i + \frac{t_0 - t_i}{t_{i+1} - t_i}(y_{i+1} - y_i)$$

input	meaning	condition
`t`	the times (strictly increasing)	
`y`	the values at those times	
`t0`	the time of the missing value	

output	meaning
`value`	the interpolated value at t0

- **Method:** $y_i + (t_0 - t_i)$(y_{$i + 1$} - y_i)/(t_{$i + 1$} - t_i) with Missing superscript or subscript argument {$i + 1$}
- **Assumes:** times strictly increasing; t0 within the observed times (no extrapolation)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
time_series_ai999x.solve('linear_interpolation', t='1, 2, 3, 4', y='10, 14, 13, 18', t0=2.5)
# -> value = 13.5   (SOLVED)
```

```
time_series_ai999x.solve('linear_interpolation', t=[0, 5, 10], y=[3.2, 4.7, 4.1], t0=8)
# -> value = 4.34   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Linearly interpolate the value at t = 2.5 from times [1, 2, 3, 4] and values [10, 14, 13, 18].	SOLVED	value = 13.5
Interpolate the missing value at t = 8 given times [0, 5, 10] and values [3.2, 4.7, 4.1].	SOLVED	value = 4.34
Interpolate the value at t = 12 given times [0, 5, 10] and values [3.2, 4.7, 4.1].	INVALID_INPUT	Linear interpolation of a missing value: t0 must lie between the first and last time (no extrapolation).

Exponential (log-linear) interpolation — `exponential_interpolation`

Fill a missing value at time t0 assuming constant growth between the neighbouring observations (linear interpolation of the logarithms).

$$\hat{y}(t_0) = y_i \left(\frac{y_{i+1}}{y_i} \right)^{\frac{t_0 - t_i}{t_{i+1} - t_i}}$$

input	meaning	condition
`t`	the times (strictly increasing)	
`y`	the values at those times	
`t0`	the time of the missing value	

output	meaning
`value`	the interpolated value at t0

- **Method:** y_i (y_{$i+1$}/y_i)^((t0 - t_i)/(t_{$i+1$} - t_i))
- **Assumes:** positive values; times strictly increasing; t0 within the observed times
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
time_series_ai999x.solve('exponential_interpolation', t='1, 2, 3, 4', y='100, 120, 150, 170',
t0=2.5)
# -> value = 134.164   (SOLVED)
```

```
time_series_ai999x.solve('exponential_interpolation', t=[2000, 2010], y=[6.1, 6.9], t0=2004)
# -> value = 6.40822   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Exponentially interpolate the value at t = 2.5 from times [1, 2, 3, 4] and values [100, 120, 150, 170].	SOLVED	value = 134.164
Use exponential interpolation for the value at t = 2004 given years [2000, 2010] and values [6.1, 6.9].	SOLVED	value = 6.40822
Exponentially interpolate the value at t = 1.5 from times [1, 2] and values [-1, 4].	INVALID_INPUT	Exponential (log-linear) interpolation: the values must contain positive values only.

Single change point in the mean (least squares) — `change_point_mean_shift`

The single change point that minimises the total within-segment sum of squares (a shift in the mean), with the means before and after.

$$\hat{k} = \arg\min_{k} \left[\sum_{t \le k} (x_t - \bar{x}_{1:k})^2 + \sum_{t > k} (x_t - \bar{x}_{k+1:n})^2 \right]$$

input	meaning	condition
`x`	the series, oldest first	
`min_size`	minimum segment length (default 2)	

output	meaning
`change_index`	k: the first segment is x_1..x_k
`mean_before`	mean of x_1..x_k
`mean_after`	mean of x_{$k+1$}..x_n
`cost`	total within-segment sum of squares
`reduction`	sum of squares removed by the split

- **Method:** exhaustive search over every admissible split
- **Assumes:** each segment has at least min_size values; ties: the smallest k
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
time_series_ai999x.solve('change_point_mean_shift', x='5, 6, 5, 7, 6, 12, 13, 12, 14')
# -> change_index = 5, mean_before = 5.8, mean_after = 12.75, cost = 5.55, reduction = 107.339
(SOLVED)
```

```
time_series_ai999x.solve('change_point_mean_shift', x=[1.2, 0.8, 1.1, 3.9, 4.2, 4.0, 4.4],
min_size=3)
# -> change_index = 3, mean_before = 1.03333, mean_after = 4.125, cost = 0.234167, reduction =
16.3858   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the change point in the mean of [5, 6, 5, 7, 6, 12, 13, 12, 14].	SOLVED	change_index = 5, mean_before = 5.8, mean_after = 12.75, cost = 5.55, reduction = 107.339
Locate the mean shift with minimum segment length 3 in [1.2, 0.8, 1.1, 3.9, 4.2, 4.0, 4.4].	SOLVED	change_index = 3, mean_before = 1.03333, mean_after = 4.125, cost = 0.234167, reduction = 16.3858
Find the change point in the mean of [5, 6, 7].	INVALID_INPUT	Single change point in the mean (least squares): the series is too short for two segments of the minimum length.

Two change points in the mean (optimal segmentation) — `two_change_points`

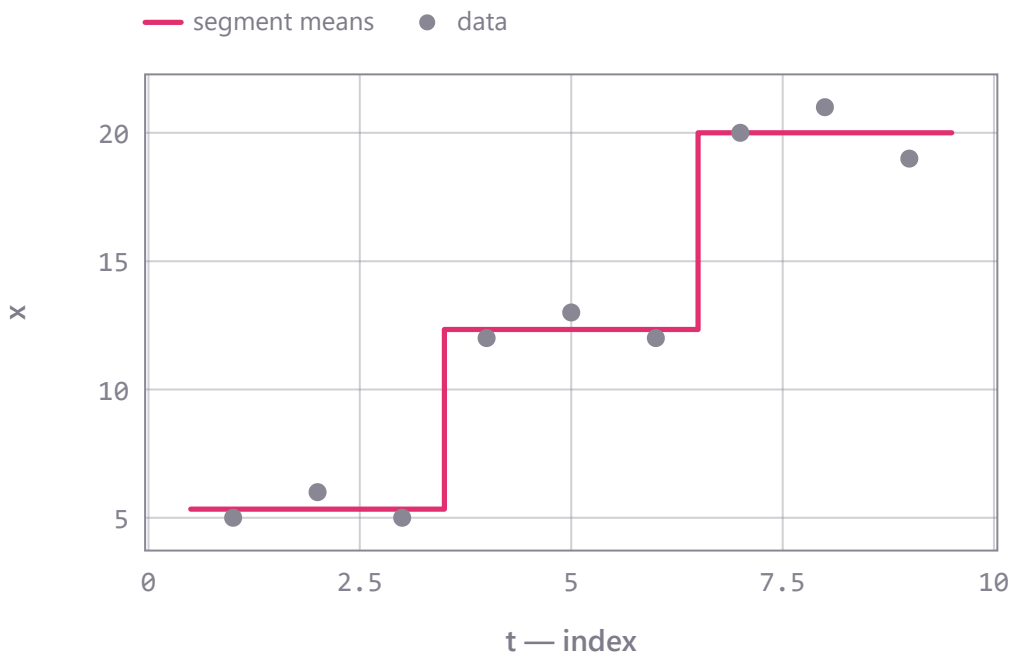
The two change points that minimise the total within-segment sum of squares (three mean levels), found by exhaustive dynamic search.

$$\min_{i<j} \ \mathrm{SS}(1{:}i) + \mathrm{SS}(i{+}1{:}j) + \mathrm{SS}(j{+}1{:}n)$$

input	meaning	condition
`x`	the series, oldest first	
`min_size`	minimum segment length (default 2)	

output	meaning
`change_1`	end of the first segment
`change_2`	end of the second segment
`means`	the three segment means
`cost`	total within-segment sum of squares

- **Method:** exhaustive search over all pairs of admissible splits
- **Assumes:** each segment has at least min_size values; ties: the smallest first, then second index
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The two change points the package finds in x = 5, 6, 5, 12, 13, 12, 20, 21, 19: after observations 3 and 6, splitting the series into segments with means 5.33333, 12.3333, 20 (total squared error 3.33333).

```
time_series_ai999x.solve('two_change_points', x='5, 6, 5, 12, 13, 12, 20, 21, 19')
# -> change_1 = 3, change_2 = 6, means = [5.333333333333333, 12.333333333333334, 20.0], cost =
3.33333   (SOLVED)
```

```
time_series_ai999x.solve('two_change_points', x=[1.0, 1.2, 0.9, 4.1, 3.8, 4.3, 4.0, 1.5, 1.1, 1.3],
min_size=3)
# -> change_1 = 3, change_2 = 7, means = [1.0333333333333334, 4.05, 1.3], cost = 0.256667
(SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find two change points in the mean of [5, 6, 5, 12, 13, 12, 20, 21, 19].	SOLVED	change_1 = 3, change_2 = 6, means = [5.333333333333333, 12.333333333333334, 20.0], cost = 3.33333
Locate two mean shifts with minimum segment length 3 in [1.0, 1.2, 0.9, 4.1, 3.8, 4.3, 4.0, 1.5, 1.1, 1.3].	SOLVED	change_1 = 3, change_2 = 7, means = [1.0333333333333334, 4.05, 1.3], cost = 0.256667

Multiple change points with a penalty (PELT) — `penalised_change_points`

The segmentation minimising the within-segment sum of squares plus a penalty per change point (optimal partitioning, as found by PELT): the number and positions of the changes in the mean.

$$\min_{K,\tau} \sum_{i=0}^{K} \mathrm{SS}(\tau_i{+}1{:}\tau_{i+1}) + \beta K$$

input	meaning	condition
`x`	the series, oldest first	
`penalty`	penalty per change point (> 0)	

output	meaning
`n_changes`	number of change points
`change_indices`	ends of all segments but the last
`cost`	within-segment sum of squares of the chosen segmentation

- **Method:** optimal partitioning F(b) = min_a F(a) + SS($a+1$:b) + penalty
- **Assumes:** penalty > 0; segments of any length >= 1; ties: the earlier last change
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
time_series_ai999x.solve('penalised_change_points', x='5, 6, 5, 12, 13, 12, 20, 21, 19, 20',
penalty=10)
# -> n_changes = 2, change_indices = [3, 6], cost = 3.33333   (SOLVED)
```

```
time_series_ai999x.solve('penalised_change_points', x=[1.0, 1.2, 0.9, 1.1, 1.3, 1.0], penalty=5)
# -> n_changes = 0, change_indices = [], cost = 0.108333   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the change points with PELT and penalty 10 in [5, 6, 5, 12, 13, 12, 20, 21, 19, 20].	SOLVED	n_changes = 2, change_indices = [3, 6], cost = 3.33333
Run penalized change point detection with penalty 5 on [1.0, 1.2, 0.9, 1.1, 1.3, 1.0].	SOLVED	n_changes = 0, change_indices = [], cost = 0.108333
Find the change points with PELT in [5, 6, 5, 12].	NEEDS_INPUT	penalised_change_points needs penalty (penalty per change point (> 0)). Example: solve('penalised_change_points', x='5,

CUSUM estimate of a change point — `cusum_change_point`

The change point estimated by the cumulative sum of deviations from the overall mean, S_k = sum_{$t \le k$}(x_t - xbar): the k with the largest |S_k|.

$$S_k = \sum_{t=1}^{k} (x_t - \bar{x}), \quad \hat{k} = \arg\max_k |S_k|$$

input	meaning	condition
`x`	the series, oldest first	

output	meaning
`change_index`	k with the largest |S_k| (first segment x_1..x_k)
`max_cusum`	max |S_k|
`mean_before`	mean of x_1..x_k
`mean_after`	mean of x_{$k+1$}..x_n

- **Method:** running sum of deviations from the mean; argmax of its absolute value
- **Assumes:** $k = 1..n-1$; ties: the smallest k
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
time_series_ai999x.solve('cusum_change_point', x='5, 6, 5, 7, 6, 12, 13, 12, 14')
# -> change_index = 5, max_cusum = 15.4444, mean_before = 5.8, mean_after = 12.75   (SOLVED)
```

```
time_series_ai999x.solve('cusum_change_point', x=[20.1, 19.8, 20.3, 17.2, 16.9, 17.4, 17.0])
# -> change_index = 3, max_cusum = 5.04286, mean_before = 20.0667, mean_after = 17.125   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Estimate the change point with CUSUM for [5, 6, 5, 7, 6, 12, 13, 12, 14].	SOLVED	change_index = 5, max_cusum = 15.4444, mean_before = 5.8, mean_after = 12.75
Use the cumulative sum of deviations to locate the shift in [20.1, 19.8, 20.3, 17.2, 16.9, 17.4, 17.0].	SOLVED	change_index = 3, max_cusum = 5.04286, mean_before = 20.0667, mean_after = 17.125

Tabular CUSUM control chart — `tabular_cusum`

The upper and lower tabular CUSUM statistics for a target mean mu0 with allowance k, and the first time either exceeds the decision interval h.

$$C_t^+ = \max(0,\, C_{t-1}^+ + x_t - \mu_0 - k), \quad C_t^- = \max(0,\, C_{t-1}^- + \mu_0 - k - x_t)$$

input	meaning	condition
`x`	the observations, oldest first	
`mu0`	target (in-control) mean	

input	meaning	condition
`k`	allowance (reference value) $k \geq 0$	
`h`	decision interval $h > 0$	

output	meaning
`c_plus`	final upper CUSUM C+_n
`c_minus`	final lower CUSUM C-_n
`alarm_index`	first t with $C+_t > h$ or $C-_t > h$ (0 if none)
`side`	upper, lower, both or none

- **Method:** C+_t = max(0, C+_{$t-1$} + x_t - mu0 - k); C-_t = max(0, C-_{$t-1$} + mu0 - k - x_t)
- **Assumes:** $C+_0 = C-_0 = 0$; an alarm needs a value strictly above h
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
time_series_ai999x.solve('tabular_cusum', x='10.2, 9.8, 10.5, 11.2, 11.8, 12.1, 11.9', mu0=10,
k=0.5, h=4)
# -> c_plus = 5, c_minus = 0, alarm_index = 7, side = upper   (SOLVED)
```

```
time_series_ai999x.solve('tabular_cusum', x=[50, 49, 48, 47.5, 47, 46.8], mu0=50, k=1, h=5)
# -> c_plus = 0, c_minus = 6.7, alarm_index = 6, side = lower   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Run a tabular CUSUM with target 10, allowance 0.5 and decision interval 4 on [10.2, 9.8, 10.5, 11.2, 11.8, 12.1, 11.9].	SOLVED	c_plus = 5, c_minus = 0, alarm_index = 7, side = upper
Compute the CUSUM statistics with target mean 50, k = 1 and h = 5 for [50, 49, 48, 47.5, 47, 46.8].	SOLVED	c_plus = 0, c_minus = 6.7, alarm_index = 6, side = lower
Run a tabular CUSUM with target 10 and allowance 0.5 on [10, 11].	NEEDS_INPUT	tabular_cusum needs h (decision interval h > 0). Example: solve('tabular_cusum', x='10.2, 9.8, 10.5, 11.2, 11.8, 12.1, 1

Kalman measurement update (scalar) — `kalman_update`

Combine a prior estimate (mean, variance) with a measurement (value, noise variance): the posterior mean and variance and the Kalman gain.

$$K = \frac{P}{P+R},\ \hat{x}^+ = \hat{x} + K(z - \hat{x}),\ P^+ = (1-K)P$$

input	meaning	condition
`m`	prior mean	
`p`	prior variance (> 0)	
`z`	the measurement	
`r`	measurement noise variance (> 0)	

output	meaning
`mean`	posterior mean
`variance`	posterior variance
`gain`	Kalman gain p/(p+r)

- **Method:** $K = p/(p+r)$; m' = m + K(z - m); p' = $(1-K)$ p
- **Assumes:** $p > 0$, $r > 0$; the measurement observes the state directly $(H = 1)$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
time_series_ai999x.solve('kalman_update', m=10, p=4, z=12, r=1)
# -> mean = 11.6, variance = 0.8, gain = 0.8   (SOLVED)
```

```
time_series_ai999x.solve('kalman_update', m=-3.5, p=0.5, z=-2.0, r=2.5)
# -> mean = -3.25, variance = 0.416667, gain = 0.166667   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Do a Kalman update with prior mean = 10, prior variance = 4, measurement = 12 and measurement variance = 1.	SOLVED	mean = 11.6, variance = 0.8, gain = 0.8
Kalman filter: prior estimate -3.5, prior variance 0.5, measurement -2.0, measurement noise variance 2.5. Find the posterior.	SOLVED	mean = -3.25, variance = 0.416667, gain = 0.166667
Do a Kalman update with prior mean = 10, prior variance = -4, measurement = 12 and measurement variance = 1.	INVALID_INPUT	Kalman measurement update (scalar): the prior variance must be positive.

One Kalman filter step (random-walk model) — `kalman_step`

One predict-and-update step of the scalar Kalman filter for a random-walk state
Missing superscript or subscript argument{$t-1$} + w_t observed as $z_t = x_t + v_t$.

$$P^- = P + Q,\ K = \frac{P^-}{P^- + R},\ \hat{x} = \hat{x}^- + K(z - \hat{x}^-),\ P = (1-K)P^-$$

input	meaning	condition
`m`	previous estimate	
`p`	previous estimate variance	
`z`	the new measurement	
`q`	process noise variance	
`r`	measurement noise variance	

output	meaning
`mean`	updated estimate
`variance`	updated variance
`gain`	Kalman gain

output	meaning
`prior_variance`	predicted variance p + q

- **Method:** predict P- = P + Q; $K = P$-/(P- + R); $x = x + K$(z - x); $P = (1 - K)$ P-
- **Assumes:** p, $q \geq 0$, $r > 0$; $F = H = 1$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
time_series_ai999x.solve('kalman_step', m=10, p=4, z=12, q=1, r=2)
# -> mean = 11.4286, variance = 1.42857, gain = 0.714286, prior_variance = 5   (SOLVED)
```

```
time_series_ai999x.solve('kalman_step', m=0.5, p=0.2, z=1.4, q=0.05, r=0.3)
# -> mean = 0.909091, variance = 0.136364, gain = 0.454545, prior_variance = 0.25   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Do one Kalman filter step with previous estimate = 10, previous variance = 4, process variance = 1, measurement = 12 and measurement variance = 2.	SOLVED	mean = 11.4286, variance = 1.42857, gain = 0.714286, prior_variance = 5
Kalman filter step: previous estimate 0.5, previous variance 0.2, process noise variance 0.05, new measurement 1.4, measurement variance 0.3.	SOLVED	mean = 0.909091, variance = 0.136364, gain = 0.454545, prior_variance = 0.25
Do one Kalman filter step with previous estimate = 10, process variance = 1, measurement = 12 and measurement variance = 2.	NEEDS_INPUT	kalman_step needs p (previous estimate variance). Example: solve('kalman_step', m=10, p=4, z=12, q=1, r=2).

Kalman filter for a random-walk (local level) model — `kalman_filter_local_level`

Filter a sequence of measurements with the scalar Kalman filter for Missing superscript or subscript argument{ $t - 1$} + w_t, $z_t = x_t + v_t$ (predict, then update, at every step): the final estimate, variance, gain, all filtered estimates and the Gaussian log-likelihood of the one-step prediction errors.

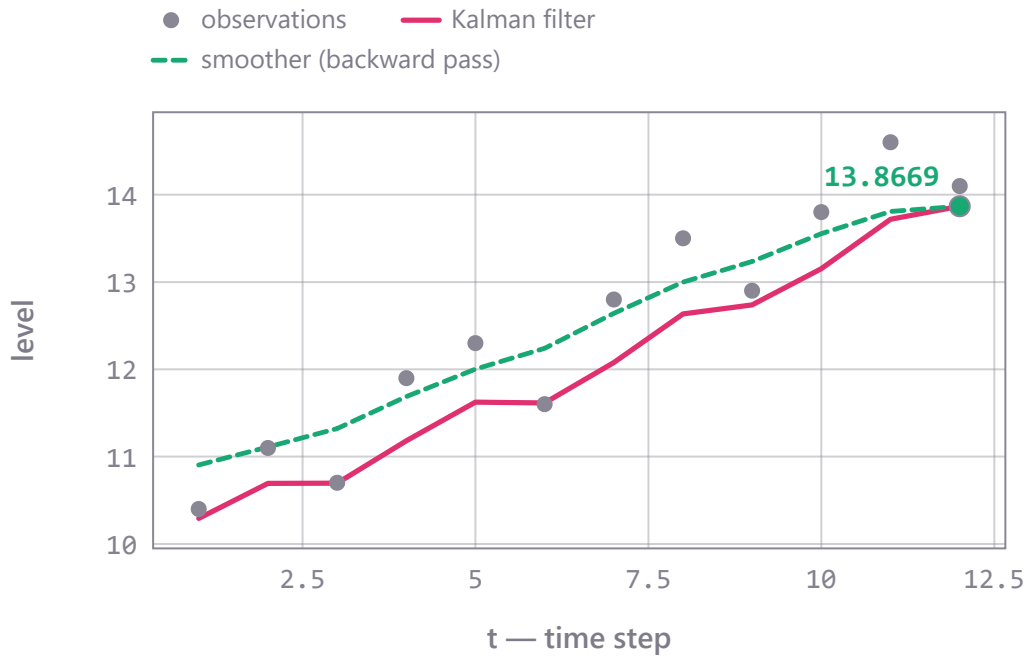
$$P_t^- = P_{t-1} + q,\ K_t = \frac{P_t^-}{P_t^- + r},\ \hat{x}_t = \hat{x}_{t-1} + K_t(z_t - \hat{x}_{t-1})$$

input	meaning	condition
`x`	the measurements, oldest first	
`q`	process noise variance	
`r`	measurement noise variance	
`m0`	initial state estimate	
`p0`	initial variance	

output	meaning
`estimate`	final filtered estimate
`variance`	final variance
`gain`	final gain

output	meaning
`estimates`	filtered estimates for every measurement
`log_likelihood`	sum of log N(v_t; 0, S_t)

- **Method:** P- = P + q; $S = P$- + r; $K = P$-/S; $x = x + K$(z - x); $P = (1 - K)$ P-
- **Assumes:** $q \geq 0$, $r > 0$, $p_0 \geq 0$; the initial estimate is the state before the first measurement
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



A local-level model (state noise q = 0.5, observation noise r = 2, start m_0 = 10, p_0 = 5) run by the package on 12 observations: the filtered level after each observation and the smoothed level from the backward pass; marked: t = 12, filtered level = 13.8669 (variance 0.780786, gain 0.390393).

```
time_series_ai999x.solve('kalman_filter_local_level', x='10.4, 11.1, 10.7, 11.9, 12.3', q=0.5, r=2,
m0=10, p0=5)
# -> estimate = 11.6241, variance = 0.79064, gain = 0.39532, estimates = [10.293333333333333,
10.693277310924369, 10.696149217809866, 11.182224215246636, 11.624103148133084], log_likelihood =
-8.61712   (SOLVED)
```

```
time_series_ai999x.solve('kalman_filter_local_level', x=[0.2, -0.1, 0.4, 0.3], q=0.01, r=0.25,
m0=0, p0=1)
# -> estimate = 0.198267, variance = 0.0677833, gain = 0.271133, estimates = [0.16031746031746033,
0.04135493880365457, 0.16042256829770477, 0.1982666548132646], log_likelihood = -2.66676   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Run a Kalman filter with process variance = 0.5, measurement variance = 2, initial estimate = 10 and initial variance = 5 for the measurements [10.4, 11.1, 10.7, 11.9, 12.3].	SOLVED	estimate = 11.6241, variance = 0.79064, gain = 0.39532, estimates = [10.293333333333333, 10.693277310924369, 10.696149217809866, 11.182224215246636, 11.624103148133084], log_likelihood = -8.61712
Apply the Kalman filter with process variance 0.01, measurement variance 0.25, initial	SOLVED	estimate = 0.198267, variance = 0.0677833, gain = 0.271133, estimates = [0.16031746031746033, 0.04135493880365457,

question	outcome	result
estimate 0 and initial variance 1 to the measurements [0.2, -0.1, 0.4, 0.3].		0.16042256829770477, 0.1982666548132646], log_likelihood = -2.66676
Run a Kalman filter with process variance = 0.5 for the measurements [1, 2].	NEEDS_INPUT	kalman_filter_local_level needs r, m0, p0 (measurement noise variance; initial state estimate; initial variance). Exampl

Rauch-Tung-Striebel smoother (random-walk model) — `rts_smoother_local_level`

Smoothed state estimates of the random-walk (local level) model: the Kalman filter forward, then the Rauch-Tung-Striebel recursion backward.

$$\hat{x}_{t|n} = \hat{x}_t + C_t(\hat{x}_{t+1|n} - \hat{x}_t),\ C_t = \frac{P_t}{P_t + q}$$

input	meaning	condition
`x`	the measurements, oldest first	
`q`	process noise variance	
`r`	measurement noise variance	
`m0`	initial state estimate	
`p0`	initial variance	

output	meaning
`smoothed`	smoothed estimates for every time
`first`	smoothed estimate at time 1
`first_variance`	its variance

- **Method:** $C_t = P_t/(P_t + q)$; $xs_t = x_t + C_t$ (xs_{t + 1} - x_t); $Ps_t = P_t + C_t^2$ (Ps_{t + 1} - P_t - q)
- **Assumes:** $q > 0$, $r > 0$, $p_0 \geq 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
time_series_ai999x.solve('rts_smoother_local_level', x='10.4, 11.1, 10.7, 11.9, 12.3', q=0.5, r=2,
m0=10, p0=5)
# -> smoothed = [10.83039127959001, 11.013479215813877, 11.174936955991214, 11.455128935166355,
11.624103148133084], first = 10.8304, first_variance = 0.694975   (SOLVED)
```

```
time_series_ai999x.solve('rts_smoother_local_level', x=[0.2, -0.1, 0.4, 0.3], q=0.01, r=0.25, m0=0,
p0=1)
# -> smoothed = [0.1808702742728623, 0.1818958800385576, 0.1941973210057952, 0.1982666548132646],
first = 0.18087, first_variance = 0.0661248   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Apply the RTS smoother with process variance = 0.5, measurement variance = 2, initial estimate = 10 and initial variance = 5 for the measurements [10.4, 11.1, 10.7, 11.9, 12.3].	SOLVED	smoothed = [10.83039127959001, 11.013479215813877, 11.174936955991214, 11.455128935166355, 11.624103148133084], first = 10.8304, first_variance = 0.694975

question	outcome	result
Smooth the measurements [0.2, -0.1, 0.4, 0.3] with the Rauch-Tung-Striebel smoother, process variance 0.01, measurement variance 0.25, initial estimate 0 and initial variance 1.	SOLVED	smoothed = [0.1808702742728623, 0.1818958800385576, 0.1941973210057952, 0.1982666548132646], first = 0.18087, first_variance = 0.0661248

Steady-state Kalman gain (random-walk model) — `kalman_steady_state_gain`

The limiting Kalman gain and variances of the random-walk (local level) model with process variance q and measurement variance r (the solution of the scalar Riccati equation).

$$P^- = \frac{q + \sqrt{q^2 + 4qr}}{2}, \quad K = \frac{P^-}{P^- + r}$$

input	meaning	condition
`q`	process noise variance (> 0)	
`r`	measurement noise variance (> 0)	

output	meaning
`gain`	steady-state gain
`prior_variance`	steady-state predicted variance P-
`posterior_variance`	steady-state filtered variance

- **Method:** P- = (q + sqrt($q^2 + 4$ q r))/2, $K = P$-/(P- + r)
- **Assumes:** $q > 0$, $r > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
time_series_ai999x.solve('kalman_steady_state_gain', q=1, r=4)
# -> gain = 0.390388, prior_variance = 2.56155, posterior_variance = 1.56155   (SOLVED)
```

```
time_series_ai999x.solve('kalman_steady_state_gain', q=0.05, r=2.5)
# -> gain = 0.131774, prior_variance = 0.379436, posterior_variance = 0.329436   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the steady-state Kalman gain with process variance 1 and measurement variance 4.	SOLVED	gain = 0.390388, prior_variance = 2.56155, posterior_variance = 1.56155
What is the steady-state Kalman gain for a random walk with process noise variance 0.05 and measurement noise variance 2.5?	SOLVED	gain = 0.131774, prior_variance = 0.379436, posterior_variance = 0.329436
Find the steady-state Kalman gain with process variance 1.	NEEDS_INPUT	kalman_steady_state_gain needs r (measurement noise variance (> 0)). Example: solve('kalman_steady_state_gain', q=1, r=4

Kalman filter for a constant-velocity model — `kalman_filter_constant_velocity`

Track position and velocity from noisy position measurements with the constant-velocity Kalman filter (discrete white-noise acceleration with variance q, time step dt): final state and covariance.

$$F = \begin{pmatrix} 1 & \Delta t \\ 0 & 1 \end{pmatrix},\ Q = q \begin{pmatrix} \Delta t^4/4 & \Delta t^3/2 \\ \Delta t^3/2 & \Delta t^2 \end{pmatrix}$$

input	meaning	condition
`x`	the measured positions, oldest first	
`dt`	time step	
`q`	acceleration (process) noise variance	
`r`	position measurement noise variance	
`p0`	initial position estimate	
`v0`	initial velocity estimate	
`var0`	initial variance of position and of velocity (uncorrelated)	

output	meaning
`position`	final position estimate
`velocity`	final velocity estimate
`p11`	final position variance
`p12`	final position-velocity covariance
`p22`	final velocity variance

- **Method:** 2-state Kalman filter recursion
- **Assumes:** F = [[1, dt], [0, 1]], H = [1, 0]; $Q = q$ [[$dt^4/4$, $dt^3/2$], [$dt^3/2$, dt^2]]; predict then update at every measurement
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
time_series_ai999x.solve('kalman_filter_constant_velocity', x='1.1, 1.9, 3.2, 4.0, 4.8', dt=1,
q=0.1, r=1, p0=0, v0=1, var0=10)
# -> position = 4.89695, velocity = 0.938562, p11 = 0.61539, p12 = 0.24431, p22 = 0.224491
(SOLVED)
```

```
time_series_ai999x.solve('kalman_filter_constant_velocity', x=[0.0, 0.6, 0.9, 1.6], dt=0.5, q=0.5,
r=0.2, p0=0, v0=0, var0=5)
# -> position = 1.51458, velocity = 0.999685, p11 = 0.13938, p12 = 0.131464, p22 = 0.273457
(SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Run a constant-velocity Kalman filter with time step 1, process variance 0.1, measurement variance 1, initial position 0, initial velocity 1 and initial variance 10 for the positions [1.1, 1.9, 3.2, 4.0, 4.8].	SOLVED	position = 4.89695, velocity = 0.938562, p11 = 0.61539, p12 = 0.24431, p22 = 0.224491

question	outcome	result
Track position and velocity with a Kalman filter: time step 0.5, acceleration variance 0.5, measurement variance 0.2, initial position 0, initial velocity 0, initial variance 5, positions [0.0, 0.6, 0.9, 1.6].	SOLVED	position = 1.51458, velocity = 0.999685, p11 = 0.13938, p12 = 0.131464, p22 = 0.273457

Steady-state Kalman gains of a constant-velocity model — `kalman_steady_state_constant_velocity`

The limiting position and velocity gains (and predicted position variance) of the constant-velocity Kalman filter with discrete white-noise acceleration q, time step dt and measurement variance r.

$$P = F\big(P - PH^{\top}(HPH^{\top} + R)^{-1}HP\big)F^{\top} + Q$$

input	meaning	condition
`dt`	time step	
`q`	acceleration noise variance (> 0)	
`r`	measurement noise variance (> 0)	

output	meaning
`k_position`	steady-state position gain
`k_velocity`	steady-state velocity gain
`prior_p11`	steady-state predicted position variance

- **Method:** iterate the Riccati recursion to its fixed point
- **Assumes:** dt, q, $r > 0$; $Q = q$ [[$dt^4/4$, $dt^3/2$], [$dt^3/2$, dt^2]]
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
time_series_ai999x.solve('kalman_steady_state_constant_velocity', dt=1, q=0.1, r=1)
# -> k_position = 0.546211, k_velocity = 0.213023, prior_p11 = 1.20367   (SOLVED)
```

```
time_series_ai999x.solve('kalman_steady_state_constant_velocity', dt=0.5, q=2, r=0.3)
# -> k_position = 0.674199, k_velocity = 0.736886, prior_p11 = 0.620808   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the steady-state Kalman gains of a constant-velocity model with time step 1, process variance 0.1 and measurement variance 1.	SOLVED	k_position = 0.546211, k_velocity = 0.213023, prior_p11 = 1.20367
What are the steady-state gains of a constant velocity Kalman filter with time step 0.5, acceleration variance 2 and measurement variance 0.3?	SOLVED	k_position = 0.674199, k_velocity = 0.736886, prior_p11 = 0.620808

Dickey-Fuller unit-root statistic — `dickey_fuller_test`

The Dickey-Fuller tau statistic: the t-ratio of gamma in the regression Delta $x_t = \gamma$ x_{$t - 1$} (+ constant, + trend) + e_t. The statistic only: compare it with Dickey-Fuller critical values.

$$\Delta x_t = \alpha + \gamma x_{t-1} + e_t, \quad \tau = \hat{\gamma}/\mathrm{se}(\hat{\gamma})$$

input	meaning	condition
`x`	the series, oldest first	
`regression`	deterministic terms: c = constant (default), ct = constant and trend, n = none	

output	meaning
`tau`	the Dickey-Fuller statistic gamma/se
`gamma`	the coefficient of x_{$t-1$}
`se`	its standard error
`nobs`	observations in the regression $(n-1)$

- **Method:** least squares of Delta x_t on x_{$t-1$} (and constant / trend t = 1..n-1)
- **Assumes:** the series is not constant; default deterministic terms: a constant; no lagged differences (see augmented_dickey_fuller_test)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
time_series_ai999x.solve('dickey_fuller_test', x='10.2, 10.8, 10.5, 11.3, 11.0, 11.9, 11.6, 12.4, 12.0, 12.9', regression='c')
# -> tau = -0.927858, gamma = -0.269585, se = 0.290546, nobs = 9   (SOLVED)
```

```
time_series_ai999x.solve('dickey_fuller_test', x=[5.0, 4.1, 4.6, 3.8, 4.4, 4.0, 4.7, 4.2, 4.5, 3.9, 4.3, 4.1], regression='ct')
# -> tau = -7.29977, gamma = -1.56586, se = 0.214508, nobs = 11   (SOLVED)
```

```
time_series_ai999x.solve('dickey_fuller_test', x=[1.2, -0.5, 0.8, -1.1, 0.3, 0.9, -0.7, 0.4, -0.2, 0.6], regression='n')
# -> tau = -6.83032, gamma = -1.59454, se = 0.233451, nobs = 9   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Compute the Dickey-Fuller statistic with a constant for [10.2, 10.8, 10.5, 11.3, 11.0, 11.9, 11.6, 12.4, 12.0, 12.9].	SOLVED	tau = -0.927858, gamma = -0.269585, se = 0.290546, nobs = 9
Find the Dickey-Fuller test statistic with constant and trend for [5.0, 4.1, 4.6, 3.8, 4.4, 4.0, 4.7, 4.2, 4.5, 3.9, 4.3, 4.1].	SOLVED	tau = -7.29977, gamma = -1.56586, se = 0.214508, nobs = 11
Compute the Dickey-Fuller statistic with no constant for [1.2, -0.5, 0.8, -1.1, 0.3, 0.9, -0.7, 0.4, -0.2, 0.6].	SOLVED	tau = -6.83032, gamma = -1.59454, se = 0.233451, nobs = 9
Compute the Dickey-Fuller statistic for [3, 3, 3, 3, 3, 3, 3, 3].	INVALID_INPUT	Dickey-Fuller unit-root statistic: the series must not be constant.

Augmented Dickey-Fuller statistic — `augmented_dickey_fuller_test`

The augmented Dickey-Fuller tau statistic with p lagged differences: the t-ratio of gamma in Delta $x_t = \gamma$ x_{$t-1$} + sum_j delta_j Delta x_{t-j} (+ constant, + trend) + e_t. The statistic only.

$$\Delta x_t = \alpha + \gamma x_{t-1} + \sum_{j=1}^{p} \delta_j \Delta x_{t-j} + e_t$$

input	meaning	condition
`x`	the series, oldest first	
`p`	number of lagged differences	
`regression`	deterministic terms: c = constant (default), ct = constant and trend, n = none	

output	meaning
`tau`	the ADF statistic
`gamma`	the coefficient of x_{$t-1$}
`se`	its standard error
`nobs`	observations in the regression $(n-1-p)$

- **Method:** least squares on $t = p+2..n$ of Delta x_t on x_{$t-1$}, Delta x_{$t-1..t-p$} (and constant / trend)
- **Assumes:** p <= n/2 - (deterministic terms) - 1; default deterministic terms: a constant; the lag length is given, not selected
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
time_series_ai999x.solve('augmented_dickey_fuller_test', x='10.2, 10.8, 10.5, 11.3, 11.0, 11.9,
11.6, 12.4, 12.0, 12.9, 12.5, 13.4', p=2, regression='c')
# -> tau = -1.18653, gamma = -0.0390526, se = 0.0329132, nobs = 9   (SOLVED)
```

```
time_series_ai999x.solve('augmented_dickey_fuller_test', x=[5.0, 4.1, 4.6, 3.8, 4.4, 4.0, 4.7, 4.2,
4.5, 3.9, 4.3, 4.1, 4.8, 4.0], p=1, regression='ct')
# -> tau = -2.14214, gamma = -1.10285, se = 0.514833, nobs = 12   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Compute the augmented Dickey-Fuller statistic with 2 lags and a constant for [10.2, 10.8, 10.5, 11.3, 11.0, 11.9, 11.6, 12.4, 12.0, 12.9, 12.5, 13.4].	SOLVED	tau = -1.18653, gamma = -0.0390526, se = 0.0329132, nobs = 9
Find the ADF statistic with 1 lag, constant and trend, for [5.0, 4.1, 4.6, 3.8, 4.4, 4.0, 4.7, 4.2, 4.5, 3.9, 4.3, 4.1, 4.8, 4.0].	SOLVED	tau = -2.14214, gamma = -1.10285, se = 0.514833, nobs = 12
Compute the augmented Dickey-Fuller statistic with 6 lags for [1, 3, 2, 4, 3, 5, 4, 6].	INVALID_INPUT	Augmented Dickey-Fuller statistic: too many lags for the series length (p <= n/2 - deterministic terms - 1).

KPSS stationarity statistic — `kpss_test`

The KPSS statistic for level (c) or trend (ct) stationarity with a Bartlett-kernel long-run variance using a given number of lags. The statistic only: compare it with KPSS critical values.

$$\mathrm{KPSS} = \frac{n^{-2}\sum_{t=1}^{n} S_t^2}{\hat{\sigma}^2_{LR}},\ \hat{\sigma}^2_{LR} = \frac{1}{n}\Big(\sum e_t^2 + 2\sum_{i=1}^{L}\big(1 - \tfrac{i}{L+1}\big)\sum_t e_t e_{t-i}\Big)$$

input	meaning	condition
`x`	the series, oldest first	
`nlags`	Bartlett lags in the long-run variance	
`regression`	c = level stationarity (default), ct = trend stationarity	

output	meaning
`kpss`	the KPSS statistic
`long_run_variance`	Bartlett long-run variance of the residuals

- **Method:** eta = sum S_t^2/n^2 over partial sums S_t of the residuals, divided by the Bartlett long-run variance
- **Assumes:** 0 <= lags < n; residuals from the mean (c) or a linear trend $t = 1..n$ (ct)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
time_series_ai999x.solve('kpss_test', x='10.2, 10.8, 10.5, 11.3, 11.0, 11.9, 11.6, 12.4', nlags=2,
regression='c')
# -> kpss = 0.401275, long_run_variance = 0.811354   (SOLVED)
```

```
time_series_ai999x.solve('kpss_test', x=[5.0, 4.1, 4.6, 3.8, 4.4, 4.0, 4.7, 4.2, 4.5, 3.9],
nlags=1, regression='ct')
# -> kpss = 0.141083, long_run_variance = 0.0574655   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Compute the KPSS statistic for level stationarity with 2 lags for [10.2, 10.8, 10.5, 11.3, 11.0, 11.9, 11.6, 12.4].	SOLVED	kpss = 0.401275, long_run_variance = 0.811354
Find the KPSS statistic for trend stationarity with 1 lag for [5.0, 4.1, 4.6, 3.8, 4.4, 4.0, 4.7, 4.2, 4.5, 3.9].	SOLVED	kpss = 0.141083, long_run_variance = 0.0574655
Compute the KPSS statistic for [1, 3, 2, 4, 3].	NEEDS_INPUT	kpss_test needs nlags (Bartlett lags in the long-run variance). Example: solve('kpss_test', x='10.2, 10.8, 10.5, 11.3, 1

Exponential smoothing forecast update — `exponential_smoothing_update`

The next forecast by simple exponential smoothing from the previous forecast, the actual value and the smoothing constant: F_new = F + alpha (A - F).

$$F_{t+1} = F_t + \alpha\,(A_t - F_t)$$

input	meaning	condition
`F`	the previous forecast	

input	meaning	condition
`A`	the actual value of that period	
`alpha`	smoothing constant, $0 < \alpha \leq 1$	

output	meaning
`forecast`	the new forecast
`error`	the forecast error A - F

- **Method:** F_new = F + alpha $(A - F) = \alpha A + (1 - \alpha)$ F
- **Assumes:** $0 < \alpha \leq 1$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
time_series_ai999x.solve('exponential_smoothing_update', F=100, A=110, alpha=0.2)
# -> forecast = 102, error = 10   (SOLVED)
```

```
time_series_ai999x.solve('exponential_smoothing_update', F=48.5, A=45, alpha=0.6)
# -> forecast = 46.4, error = -3.5   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
With exponential smoothing, the previous forecast was 100, the actual value was 110 and alpha = 0.2. What is the new forecast?	SOLVED	forecast = 102, error = 10
Using a smoothing constant of 0.6, the last forecast was 48.5 and actual demand was 45. Find the next forecast with exponential smoothing.	SOLVED	forecast = 46.4, error = -3.5
With exponential smoothing, the previous forecast was 100 and the actual value was 110. What is the new forecast?	NEEDS_INPUT	exponential_smoothing_update needs alpha (smoothing constant, 0 < alpha <= 1). Example: solve('exponential_smoothing_upd

One update of Holt's method — `holt_update`

Update Holt's level and trend with one new observation and forecast h periods ahead.

$$\ell_t = \alpha y_t + (1 - \alpha)(\ell_{t-1} + b_{t-1}),\ b_t = \beta(\ell_t - \ell_{t-1}) + (1 - \beta) b_{t-1}$$

input	meaning	condition
`l`	previous level	
`b`	previous trend	
`y`	the new observation	
`alpha`	level smoothing constant	
`beta`	trend smoothing constant	
`h`	forecast horizon (default 1)	

output	meaning
`level`	new level
`trend`	new trend
`forecast`	level + h trend

- **Method:** $l = \alpha\, y + (1 - \alpha)$(l + b); $b = \beta$ (l_new - l) + (1 - beta) b
- **Assumes:** $0 < \alpha, \beta \leq 1$; $b_t = \beta$ (l_t - l_{$t - 1$}) + $(1 - \beta)$ b_{$t - 1$}
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
time_series_ai999x.solve('holt_update', l=50, b=2, y=55, alpha=0.3, beta=0.1)
# -> level = 52.9, trend = 2.09, forecast = 54.99   (SOLVED)
```

```
time_series_ai999x.solve('holt_update', l=120.5, b=-1.5, y=116, alpha=0.5, beta=0.2, h=3)
# -> level = 117.5, trend = -1.8, forecast = 112.1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Update Holt's method with previous level 50, previous trend 2, new observation 55, alpha = 0.3 and beta = 0.1.	SOLVED	level = 52.9, trend = 2.09, forecast = 54.99
Holt's method: previous level 120.5, previous trend -1.5, new observation 116, alpha = 0.5, beta = 0.2. Forecast 3 periods ahead.	SOLVED	level = 117.5, trend = -1.8, forecast = 112.1
Update Holt's method with previous level 50, new observation 55, alpha = 0.3 and beta = 0.1.	NEEDS_INPUT	holt_update needs b (previous trend). Example: solve('holt_update', l=50, b=2, y=55, alpha=0.3, beta=0.1).

Random-walk-with-drift (drift method) forecast — `drift_forecast`

The drift method: the average change per period $(x_n - x_1)/(n - 1)$ and the forecast x_n + h x drift.

$$\hat{x}_{n+h} = x_n + h\,\frac{x_n - x_1}{n - 1}$$

input	meaning	condition
`x`	the series, oldest first	
`h`	periods ahead (default 1)	

output	meaning
`drift`	average change per period
`forecast`	x_n + h drift

- **Method:** drift = (x_n - x_1)/(n - 1)
- **Assumes:** $n \geq 2$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
time_series_ai999x.solve('drift_forecast', x='100, 104, 103, 109, 112', h=3)
# -> drift = 3, forecast = 121   (SOLVED)
```

```
time_series_ai999x.solve('drift_forecast', x=[52.0, 50.5, 49.8, 47.1])
# -> drift = -1.63333, forecast = 45.4667   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Forecast 3 periods ahead with the drift method for [100, 104, 103, 109, 112].	SOLVED	drift = 3, forecast = 121
Give the random walk with drift forecast for [52.0, 50.5, 49.8, 47.1].	SOLVED	drift = -1.63333, forecast = 45.4667
Give the drift method forecast for [5].	INVALID_INPUT	x: needs at least 2 entries, got 1.

AR(1) forecast towards the mean — `ar1_forecast`

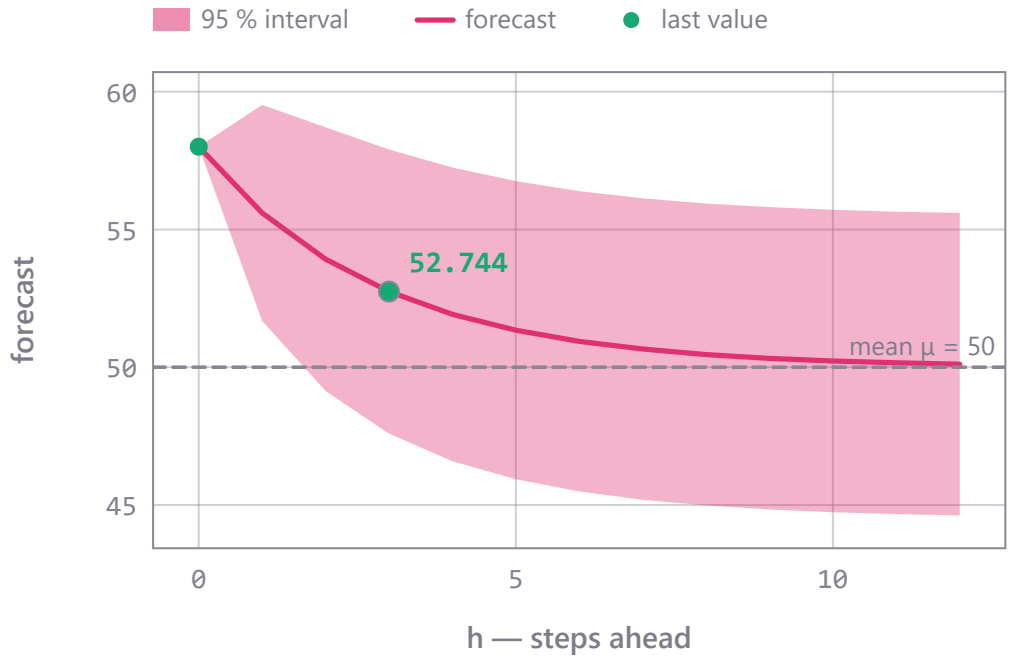
The h-step forecast $mu + \phi^h$ (x_n - mu) of a stationary AR(1) with mean mu, and (when the innovation variance is given) its forecast error variance and 95 % prediction interval.

$$\hat{x}_{n+h} = \mu + \phi^h (x_n - \mu), \ \mathrm{Var} = \sigma^2 \frac{1-\phi^{2h}}{1-\phi^2}$$

input	meaning	condition
`x`	the last observed value x_n	
`phi`	AR coefficient, |phi| < 1	
`mu`	the process mean	
`h`	forecast horizon (default 1)	
`sigma2`	innovation variance (optional, for the interval)	

output	meaning
`forecast`	$mu + \phi^h$ (x_n - mu)
`variance`	σ^2 (1 - phi^(2h)) / $(1-\phi^2)$
`lower`	lower 95 % limit
`upper`	upper 95 % limit

- **Method:** closed forms of the AR(1) recursion
- **Assumes:** |phi| < 1; known parameters; normal innovations for the interval
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



AR(1) forecasts from the last value x = 58 with φ = 0.7, μ = 50 and innovation variance σ^2 = 4, 1 to 12 steps ahead, with the 95 % interval the package returns at each step (it widens to [44.6223, 55.5992] at h = 12); marked: h = 3, forecast = 52.744.

```
time_series_ai999x.solve('ar1_forecast', x=58, phi=0.7, mu=50, h=3)
# -> forecast = 52.744   (SOLVED)
```

```
time_series_ai999x.solve('ar1_forecast', x=2.5, phi=-0.4, mu=1, h=2, sigma2=4)
# -> forecast = 1.24, variance = 4.64, lower = -2.98189, upper = 5.46189   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Forecast 3 steps ahead for an AR(1) process with mean 50, phi = 0.7 and last value 58.	SOLVED	forecast = 52.744
For an AR(1) model with mean 1, phi = -0.4 and white noise variance 4, the last value is 2.5. Forecast 2 steps ahead with a prediction interval.	SOLVED	forecast = 1.24, variance = 4.64, lower = -2.98189, upper = 5.46189
Forecast 3 steps ahead for an AR(1) process with mean 50, phi = 1.3 and last value 58.	INVALID_INPUT	AR(1) forecast towards the mean: |phi| must be less than 1 (stationarity).

Best smoothing constant among candidates — `ses_best_alpha`

Among candidate smoothing constants, the one whose simple exponential smoothing $(l_0 = x_1)$ gives the smallest sum of squared one-step errors, with that SSE and its forecast.

$$\alpha^* = \arg\min_{\alpha \in A} \sum_t (x_t - \ell_{t-1}(\alpha))^2$$

input	meaning	condition
`x`	the series, oldest first	
`alphas`	candidate smoothing constants, each in (0, 1]	

output	meaning
`alpha`	the best smoothing constant
`sse`	its sum of squared one-step errors
`forecast`	its forecast of the next period

- **Method:** simple exponential smoothing for each candidate; the smallest SSE
- **Assumes:** $l_0 = x_1$; ties: the first candidate listed
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
time_series_ai999x.solve('ses_best_alpha', x='20, 22, 21, 24, 23, 26', alphas='0.1, 0.3, 0.5')
# -> alpha = 0.5, sse = 23.8125, forecast = 24.375   (SOLVED)
```

```
time_series_ai999x.solve('ses_best_alpha', x=[5, 9, 4, 8, 6, 7, 5], alphas=[0.2, 0.8])
# -> alpha = 0.2, sse = 28.2414, forecast = 5.93542   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Which smoothing constant among [0.1, 0.3, 0.5] gives the smallest SSE for simple exponential smoothing of [20, 22, 21, 24, 23, 26]?	SOLVED	alpha = 0.5, sse = 23.8125, forecast = 24.375
Choose the best alpha from the candidates [0.2, 0.8] for exponential smoothing of the series [5, 9, 4, 8, 6, 7, 5].	SOLVED	alpha = 0.2, sse = 28.2414, forecast = 5.93542

Linear trend with seasonal dummy variables — `trend_seasonal_regression`

Fit $x_t = a + b$ t + (effect of t's season) by least squares with seasonal dummies (the first season is the baseline, effect 0; the series starts in season 1) and forecast period n + h.

$$x_t = a + bt + \sum_{j=2}^{m} \gamma_j D_{j,t} + e_t$$

input	meaning	condition
`x`	the series, oldest first, starting in season 1	
`m`	season length	
`h`	periods ahead (default 1)	

output	meaning
`forecast`	the fitted value at $t = n + h$
`intercept`	a
`slope`	b per period
`season_effects`	the m seasonal effects (season 1 = 0)

- **Method:** ordinary least squares on (1, t, D_2, ..., D_m)
- **Assumes:** $n \geq m + 3$; x_1 is in season 1; season 1 is the baseline

- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
time_series_ai999x.solve('trend_seasonal_regression', x='30, 21, 29, 31, 40, 30, 39, 41, 50, 39,
48, 51', m=4, h=1)
# -> forecast = 59.25, intercept = 27.9688, slope = 2.40625, season_effects = [0.0,
-12.406249999999996, -6.145833333333333, -6.218749999999999]   (SOLVED)
```

```
time_series_ai999x.solve('trend_seasonal_regression', x=[12, 18, 15, 14, 20, 17, 16, 22, 19], m=3,
h=2)
# -> forecast = 24, intercept = 11.3333, slope = 0.666667, season_effects = [0.0,
5.333333333333335, 1.6666666666666667]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Fit a linear trend with seasonal dummies to the quarterly series [30, 21, 29, 31, 40, 30, 39, 41, 50, 39, 48, 51] and forecast 1 period ahead.	SOLVED	forecast = 59.25, intercept = 27.9688, slope = 2.40625, season_effects = [0.0, -12.406249999999996, -6.145833333333333, -6.218749999999999]
Using a trend and seasonal dummy regression with season length 3, forecast 2 periods ahead for [12, 18, 15, 14, 20, 17, 16, 22, 19].	SOLVED	forecast = 24, intercept = 11.3333, slope = 0.666667, season_effects = [0.0, 5.333333333333335, 1.6666666666666667]
Fit a linear trend with seasonal dummies to the quarterly series [30, 21, 29].	INVALID_INPUT	x: needs at least 5 entries, got 3.

Theoretical autocorrelations of an AR(2) process — `ar2_theoretical_acf`

The autocorrelations rho_1..rho_K of a stationary AR(2) process from the Yule-Walker recursion $rho_1 = phi1/(1 - phi2)$, $rho_k = phi1$ rho_{$k-1$} + phi2 rho_{$k-2$}.

$$\rho_1 = \frac{\phi_1}{1-\phi_2},\ \rho_k = \phi_1\rho_{k-1} + \phi_2\rho_{k-2}$$

input	meaning	condition
`phi1`	phi_1	
`phi2`	phi_2	
`K`	number of lags (default 5)	

output	meaning
`acf`	rho_1..rho_K
`rho1`	rho_1

- **Method:** Yule-Walker recursion
- **Assumes:** stationary AR(2)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
time_series_ai999x.solve('ar2_theoretical_acf', phi1=0.5, phi2=0.3)
# -> acf = [0.7142857142857143, 0.6571428571428571, 0.5428571428571428, 0.4685714285714285,
0.39714285714285713], rho1 = 0.714286   (SOLVED)
```

```
time_series_ai999x.solve('ar2_theoretical_acf', phi1=1.0, phi2=-0.5, K=4)
# -> acf = [0.6666666666666666, 0.16666666666666663, -0.16666666666666669, -0.25], rho1 = 0.666667
(SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the autocorrelations of the AR(2) process with phi1 = 0.5 and phi2 = 0.3.	SOLVED	acf = [0.7142857142857143, 0.6571428571428571, 0.5428571428571428, 0.4685714285714285, 0.39714285714285713], rho1 = 0.714286
Compute the theoretical ACF up to lag 4 of the AR(2) model with phi1 = 1.0 and phi2 = -0.5.	SOLVED	acf = [0.6666666666666666, 0.16666666666666663, -0.16666666666666669, -0.25], rho1 = 0.666667
Find the autocorrelations of the AR(2) process with phi1 = 0.5 and phi2 = 0.6.	INVALID_INPUT	Theoretical autocorrelations of an AR(2) process: the AR(2) process must be stationary.

Theoretical partial autocorrelations of an MA(1) process — `ma1_theoretical_pacf`

The partial autocorrelations phi_11..phi_KK of $x_t = e_t + \theta$ e_{$t-1$} (they decay, never cut off).

$$\phi_{kk} = -\frac{(-\theta)^k(1-\theta^2)}{1-\theta^{2(k+1)}}$$

input	meaning	condition
`theta`	MA coefficient, |theta| != 1	
`K`	number of lags (default 5)	

output	meaning
`pacf`	phi_11..phi_KK
`pacf1`	$phi_1 1 = \theta/(1+\theta^2)$

- **Method:** closed form phi_kk = -$(-\theta)^k$ (1 - theta^2)/(1 - theta^(2(k+1)))
- **Assumes:** |theta| != 1; plus-sign MA convention
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
time_series_ai999x.solve('ma1_theoretical_pacf', theta=0.6)
# -> pacf = [0.441176470588, -0.241675617615, 0.140601566406, -0.0834485816626, 0.0498749669471],
pacf1 = 0.441176   (SOLVED)
```

```
time_series_ai999x.solve('ma1_theoretical_pacf', theta=-0.8, K=3)
# -> pacf = [-0.487804878049, -0.312256049961, -0.221477810692], pacf1 = -0.487805   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the partial autocorrelations of the MA(1) process with theta = 0.6.	SOLVED	pacf = [0.441176470588, -0.241675617615, 0.140601566406, -0.0834485816626, 0.0498749669471], pacf1 = 0.441176
Compute the theoretical PACF up to lag 3 of the MA(1) model with theta = -0.8.	SOLVED	pacf = [-0.487804878049, -0.312256049961, -0.221477810692], pacf1 = -0.487805
Find the partial autocorrelations of the MA(1) process with theta = 1.	INVALID_INPUT	Theoretical partial autocorrelations of an MA(1) process: |theta| = 1 is not allowed (the partial autocorrelations do no

Theoretical partial autocorrelations of an ARMA(p,q) process — `arma_theoretical_pacf`

The partial autocorrelations phi_11..phi_KK of a stationary ARMA(p,q) process (Durbin-Levinson on its exact autocorrelations).

$$\phi_{kk} = \frac{\rho_k - \sum_{j=1}^{k-1} \phi_{k-1,j}\rho_{k-j}}{1 - \sum_{j=1}^{k-1} \phi_{k-1,j}\rho_j}$$

input	meaning	condition
`phi`	AR coefficients ([0] for none)	
`theta`	MA coefficients ([0] for none)	
`K`	number of lags (default 5)	

output	meaning
`pacf`	phi_11..phi_KK
`pacf1`	$phi_1 1 = rho_1$

- **Method:** exact autocovariances, then the Durbin-Levinson recursion
- **Assumes:** stationary AR part; plus-sign MA convention
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
time_series_ai999x.solve('arma_theoretical_pacf', phi='0.5', theta='0.4')
# -> pacf = [0.6923076923076924, -0.2556818181818184, 0.1010327795240235, -0.04033486904612524,
0.016128951324258605], pacf1 = 0.692308   (SOLVED)
```

```
time_series_ai999x.solve('arma_theoretical_pacf', phi=[0.6, -0.3], theta=[0.2], K=4)
# -> pacf = [0.5515075376884422, -0.3926998491704374, 0.07804182053618533, -0.015604405193828833],
pacf1 = 0.551508   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the partial autocorrelations of the ARMA(1,1) process with phi = [0.5] and theta = [0.4].	SOLVED	pacf = [0.6923076923076924, -0.2556818181818184, 0.1010327795240235, -0.04033486904612524, 0.016128951324258605], pacf1 = 0.692308
Compute the theoretical PACF up to lag 4 of the ARMA(2,1) model with phi = [0.6, -0.3] and theta = [0.2].	SOLVED	pacf = [0.5515075376884422, -0.3926998491704374, 0.07804182053618533, -0.015604405193828833], pacf1 = 0.551508